

Statements, Functions and Program Organization

CSC 230 : C and Software Tools

NC State Department of Computer
Science

Topics for Today

- Flow-of-control
- Functions, Memory and Storage Class
- Multi-file programs
 - Global variable and function declaration
 - Header files
 - Compile and link

Flow-of-Control Statements

- C shares most flow-of-control statements with Java
- But, there are some differences.

```
for ( int i = 0; i < 25; i++ ) {  
    ...  
}
```

```
if ( x < 25 ) {  
    ...  
} else {  
    ...  
}
```

```
do {  
    ...  
} while ( !done );
```

```
while ( i != 0 ) {  
    ...  
}
```

```
switch ( code ) {  
    case 0:  
        ...;  
        break;  
    case 1:  
        ...;  
        break;  
    default:  
        ...;  
}
```

Constant Expressions

- In some places, C requires a value that can be determined at compile time
 - e.g., in the cases of a switch statement.
 - And a bunch of other places we'll talk about later
- C will let you use an expression here
 - But **the compiler** must be able to determine the value of the expression.
 - So, it can't depend on execution-time behavior
 - These are called *constant expressions*

Constant Expressions

- The compiler will reject our code if we put some other expression where a constant expression is required.

```
int three = 3;

switch ( val ) {

  case 0: // trivial
    val += 1;
    break;

  case 2 / 2: // fine
    val -= 1;
    break;

  case 25/7%2 + 19 - 6*3: // OK
    val *= 2;
    break;

  case three: // Nope
    break;
}
```

Constant Expressions

- Why?
- This is what the compiler will try to create
 - It's a jump table
 - With each index pointing to a case.

0	→
1	→
2	→

```
int three = 3;

switch ( val ) {

  case 0:
    val += 1;
    break;

  case 2 / 2:
    val -= 1;
    break;

  case 25/7%2 + 9 - 6*3:
    val *= 2;
    break;

}
```

Changing the Flow of Control

I'm probably
evil 😊

- `break`, `continue` and `goto` let us alter the normal flow of control.
- Here's a nested loop with obvious flow-of control

```
for ( int i = 0; i < 10; i++ ) {
    for ( int j = 0; j < 10; j++ ) {
        printf( "." );
        printf( "*" );
    }
    printf( "\n" );
}
```

outputs

[illegible]

Changing the Flow of Control

- `continue` lets us skip the rest of the innermost loop's current iteration.

```
for ( int i = 0; i < 10; i++ ) {
    for ( int j = 0; j < 10; j++ ) {
        printf( "." );
        if ( i + j > 12 )
            continue;
        printf( "*" );
    }
    printf( "\n" );
}
```

outputs

A large graphic composed of asterisks (*) arranged in a grid pattern to form the shape of the letter 'E'. The 'E' is formed by a solid vertical bar on the left side, with horizontal bars extending from its top, middle, and bottom. The background consists of a sparse field of asterisks.

Skip printing the stars when $i + j$ gets too big.

Changing the Flow of Control

- `break` lets us bail out of the innermost loop early (and we need it for the switch statement)

```
for ( int i = 0; i < 10; i++ ) {
    for ( int j = 0; j < 10; j++ ) {
        printf( "." );
        if ( i + j > 12 )
            break;
        printf( "*" );
    }
    printf( "\n" );
}
```

outputs

- Unlike Java, C doesn't have a labeled break.

Exit the inner loop when $i + j$ gets too big.

Changing the Flow of Control

- But, that's OK, ... err ... we have `goto`
- We can put a *label* before any statement in a function.

`myLabel:`

- Then, we can jump to that label when we need to.

`goto myLabel;`

- We can even jump in and out of loops.

Changing the Flow of Control

- With `goto`, we can jump out of loops of any depth

```
for ( int i = 0; i < 10; i++ ) {  
    for ( int j = 0; j < 10; j++ ) {  
        printf( "." );  
        if ( i + j > 12 )  
            goto done;  
        printf( "*" );  
    }  
    printf( "\n" );  
}  
done:
```

This is a destination for a goto.

This says jump to the done label right now.

outputs

```
*****  
*****  
*****  
*****  
*****
```

Exit both loops when $i + j$ gets too big.

Changing the Flow of Control

- With goto, we don't need any structured flow of control statements at all.
 - But, then we have no ... well ... structure.

```
int i = 0;
outer_top:
  if ( i >= 10 )
    goto finished;
  int j = 0;
inner_top:
  if ( j >= 10 )
    goto done_2;
  printf( "." );
  printf( "*" );
  j++;
  goto inner_top;
done_2:
  printf( "\n" );
  i++;
  goto outer_top;
finished:
```

outputs

```
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

Nested
looping with
just goto.

Yuck. ... but this is how
loops are implemented
at the machine level.

Labels and Scope

- Labels used by goto don't obey block-structured scoping rules.
- You can goto a label that hasn't occurred yet.
- You can goto a label in a more deeply nested block.
- But, you can't goto a label in another function.

```
int bake(int t, int d){  
    while ( d-- > 0 ) {  
        if ( t++ > 350 )  
            goto done;  
        printf( "%d\n", t );  
    }  
done:    ←  
    return 0;  
}
```

```
int broil(int t, int d){  
    while ( d-- > 0 ) {  
        if ( t++ > 450 )  
            goto done;  
        printf( "%d\n", t );  
    }  
done:    ←  
    return 0;  
}
```

Using and Not Using goto

- You should avoid using goto.
 - It makes it more difficult to understand loops.
 - You have to read the entire loop to see where it may terminate.
 - Maybe a reason to also avoid break & continue.
- Exceptions?
 - In some cases, it can simplify error handling.
 - In some cases, it can give you better performance.

```
if ( ... ) {  
    for ( ... ) {  
        while ( ... ) {  
            if ( ... ) {  
                ...;  
                if ( disaster )  
                    goto cleanup;  
                ...;  
            }  
        }  
    }  
}  
  
cleanup:  
    /** Clean up partial computation  
    and return */
```

Security Sidebar: “goto fail”

- iOS 7.0.6 used the following code to verify SSL messages
- If third SHA1 update operation has no error (err == 0)
 - The marked line jumps to code that returns “verified” if err == 0
 - Skips the actual sslRawVerify() method

```
. . .
hashOut.data = hashes + SSL_MD5_DIGEST_LEN;
hashOut.length = SSL_SHA1_DIGEST_LEN;
if ((err = SSLFreeBuffer(&hashCtx)) != 0)
    goto fail;
if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
    goto fail; /* MISTAKE! THIS LINE SHOULD NOT BE HERE */
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
    goto fail;

err = sslRawVerify(...);
. . .
```

Side Effects

- Other than just computing a value, expressions can have side effects.
 - Maybe you use post/pre increment/decrement
 - Maybe you call a function that
 - Reads some input
 - Changes some global variables
- Expressions with side effects can be tricky.
- Don't believe me? Just watch.

Expression Evaluation Order

- The C standard leaves some details of expression evaluation up to the compiler
 - Why? Maybe the compiler can write faster code if we give it some flexibility.
- What if part of an expression has side effects?

```
a = 2;  
b = 3;  
c = f( &a ) + g( &b ) + h( &a, &b )
```

What if these functions modify a and b?

```
a = 2;  
b = ++a + a;
```

```
x = 1;  
b = x-- && x;
```

Are we guaranteed anything here?

```
a = 2;  
b = a * a + a++ + ++a + a / a;
```

Or here?

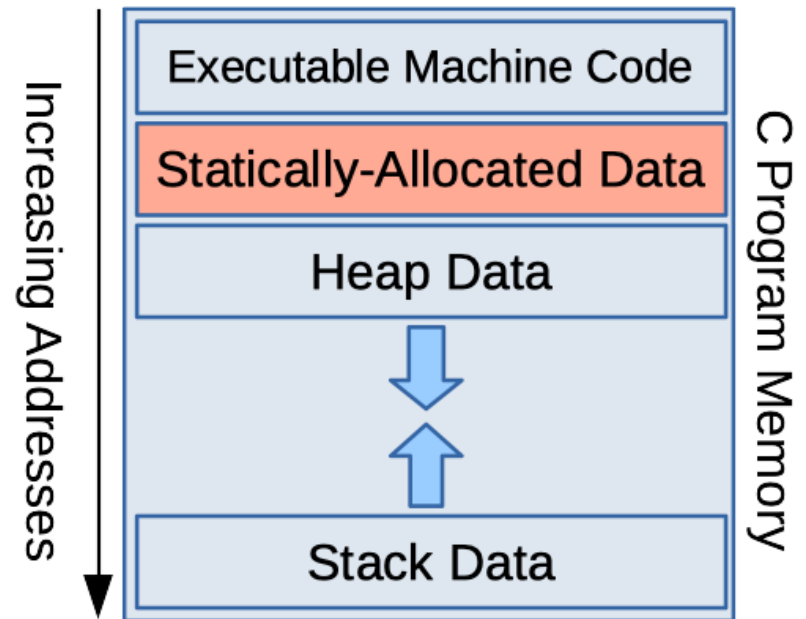
Evaluation Order

- C defines a surprisingly small number of *sequence points*
 - Places where know side effects to the left will occur before evaluation continues
- Here they are:
 - statement termination ;
 - closing parenthesis in a condition evaluation)
 - the following operators:
a && b a || b a ? b : c a , b
 - after evaluation of all arguments to a function call
 - after returning a value from a function
- In general, avoid cases where this matters
 - Don't mix side effects with multiple evaluations in the same expression.

But not between
function
parameters.

Storage Class

- A variable's *storage class* determines
 - Its lifetime
 - Where it's allocated
 - How it's initialized
- Global variables have *static storage* class.



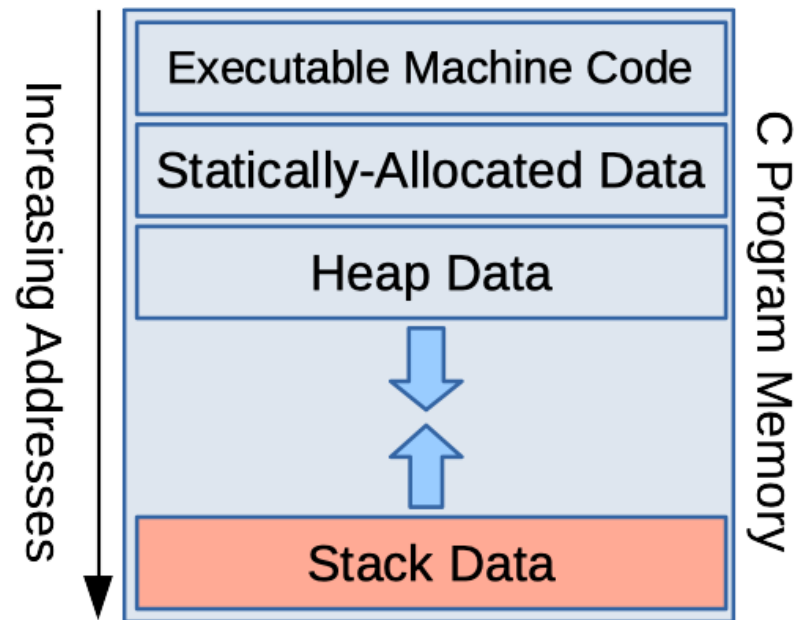
Static/Global Variable Initialization

- Variables with static allocation (e.g., globals) are initialized at program start-up.
- If you don't initialize them, they automatically get **zero**.
- If you do initialize them, you need a **constant expression**

```
char globalC = 'a';           // Yes
int globalI = 15 + (39 % 3);   // Yes
int globalJ = globalI + 1;     // Nope
```

Auto Storage Class

- Storage for local variables is **automatically** allocated
 - When control enters their containing block
 - Maybe when the function starts, it's the compiler's choice.
 - These variables have the **auto** storage class
 - We'll also call them *local* or *stack* variables.
- Auto is even a keyword:
`auto int i = 5;`



Auto Variable Initialization

- If you don't initialize them ... you get whatever's left over in that part of stack memory.

```
for ( int i = 0; i < 100; i++ ) {  
    int a;  
    ...;  
}
```

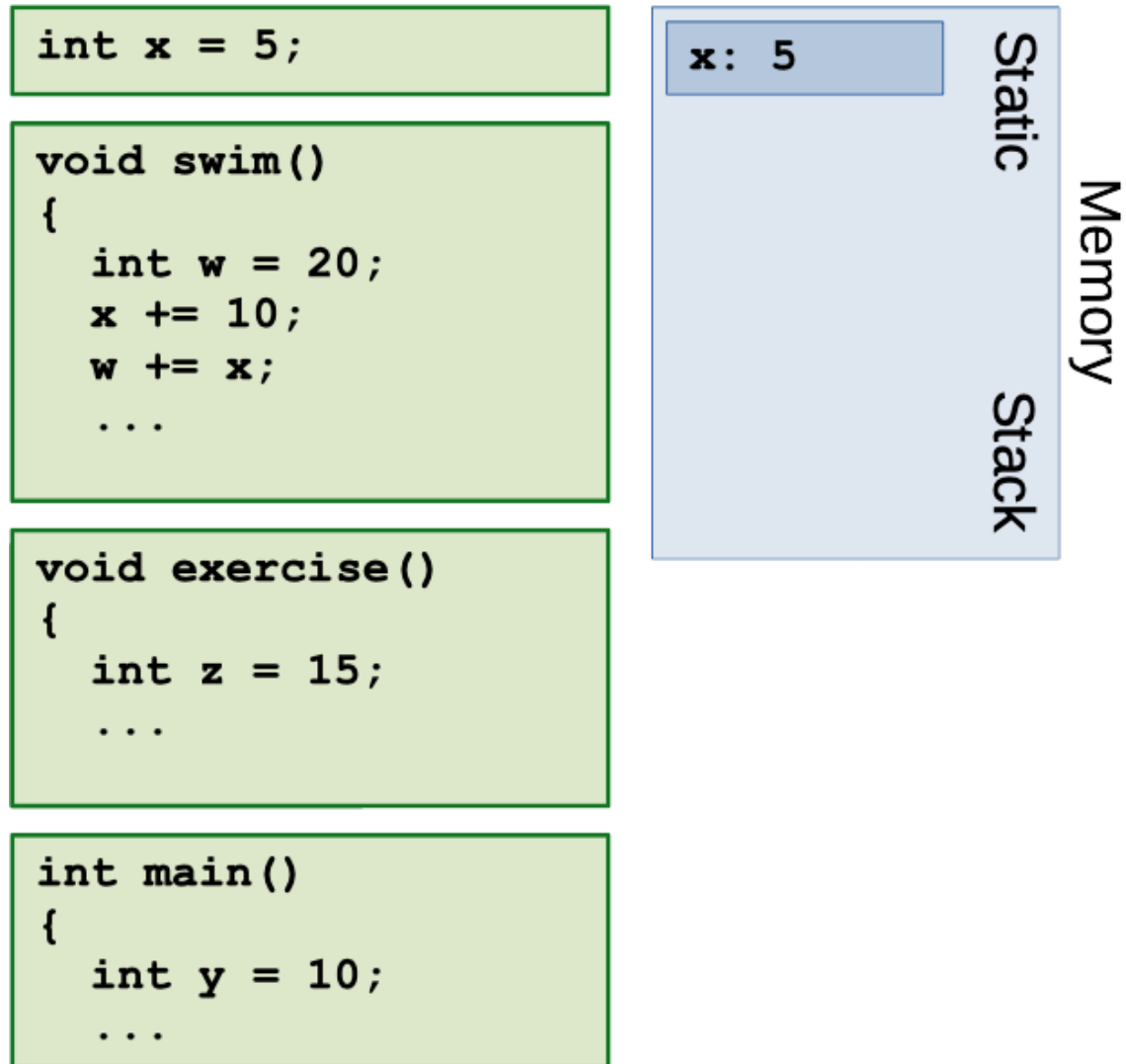
Who knows what value this will have. A common mistake.

- If you do initialize ... the initialization is evaluated each time control passes the declaration.

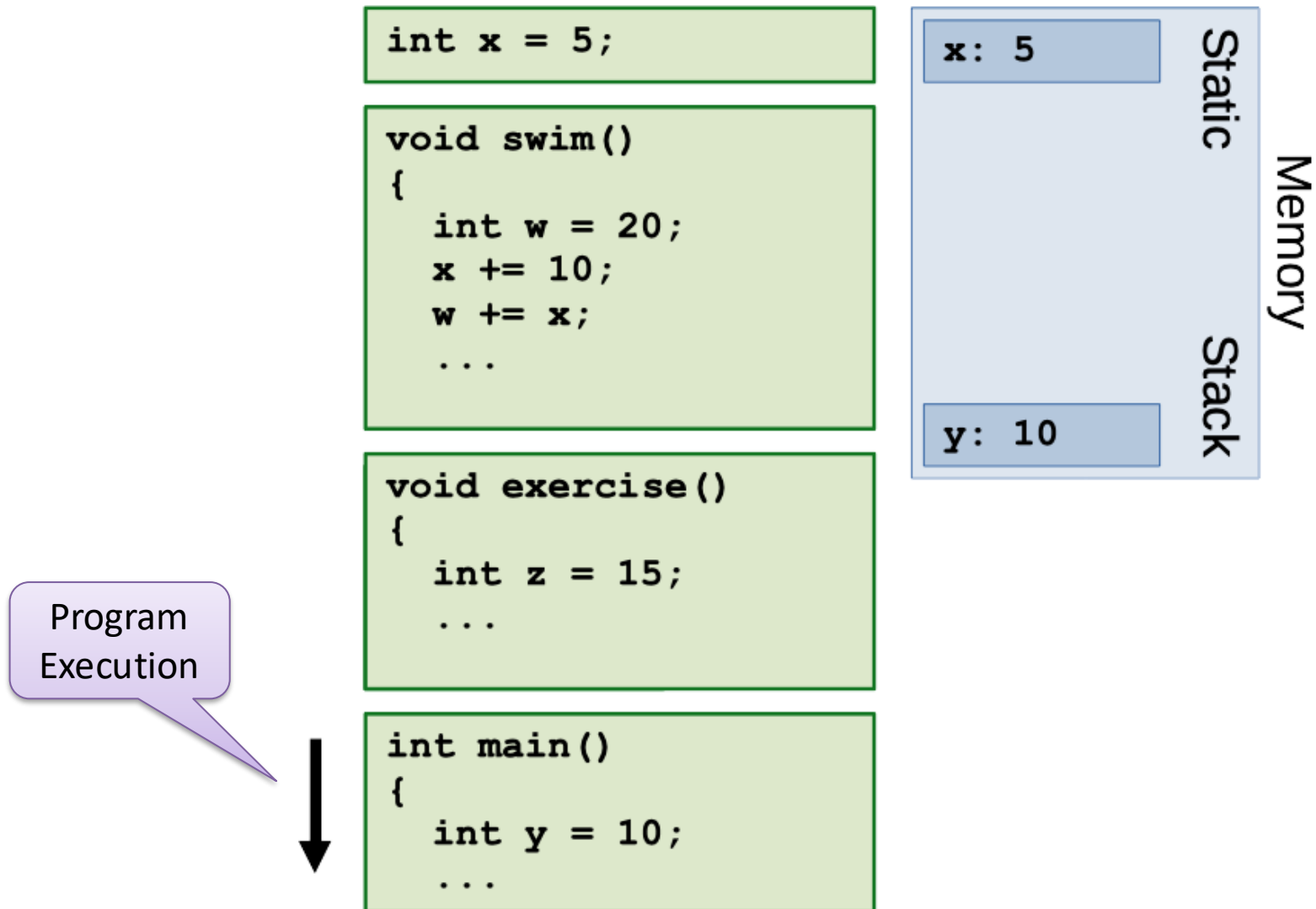
```
for ( int i = 0; i < 100; i++ ) {  
    int a = pow( 3.25, i ) + sqrt( i + 1 );  
    ...;  
}
```

This gets evaluated on every iteration.

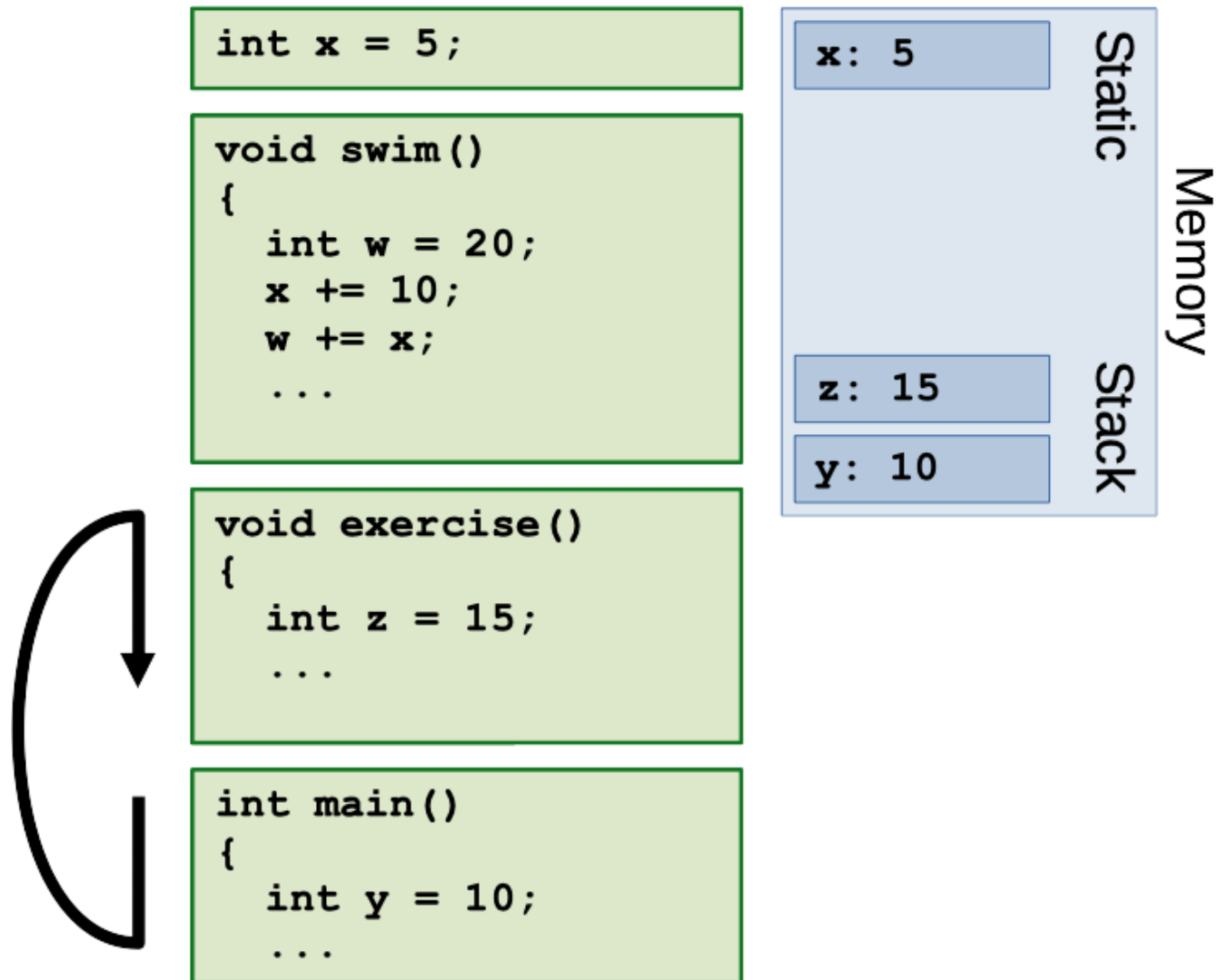
Variable Allocation/Initialization



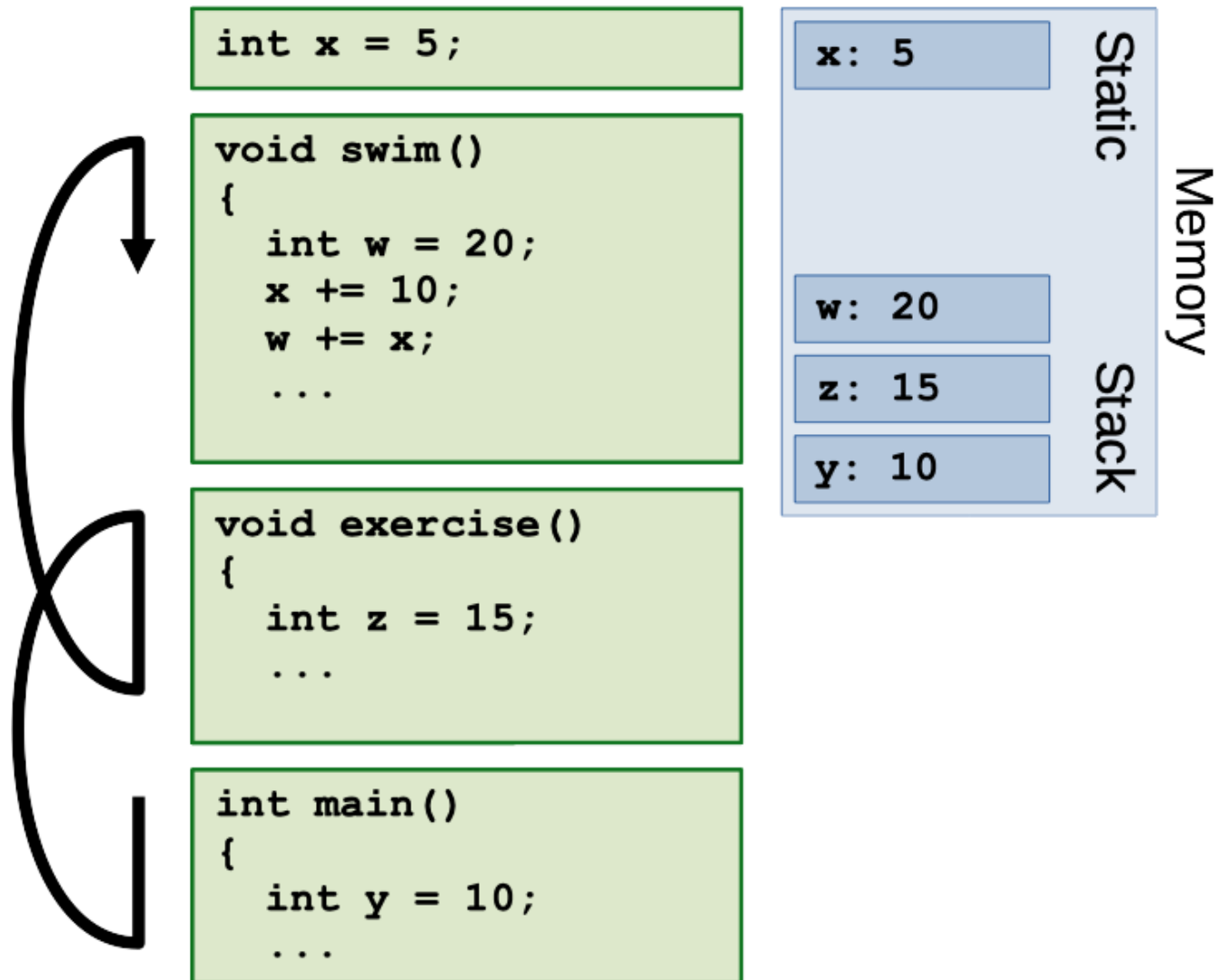
Variable Allocation/Initialization



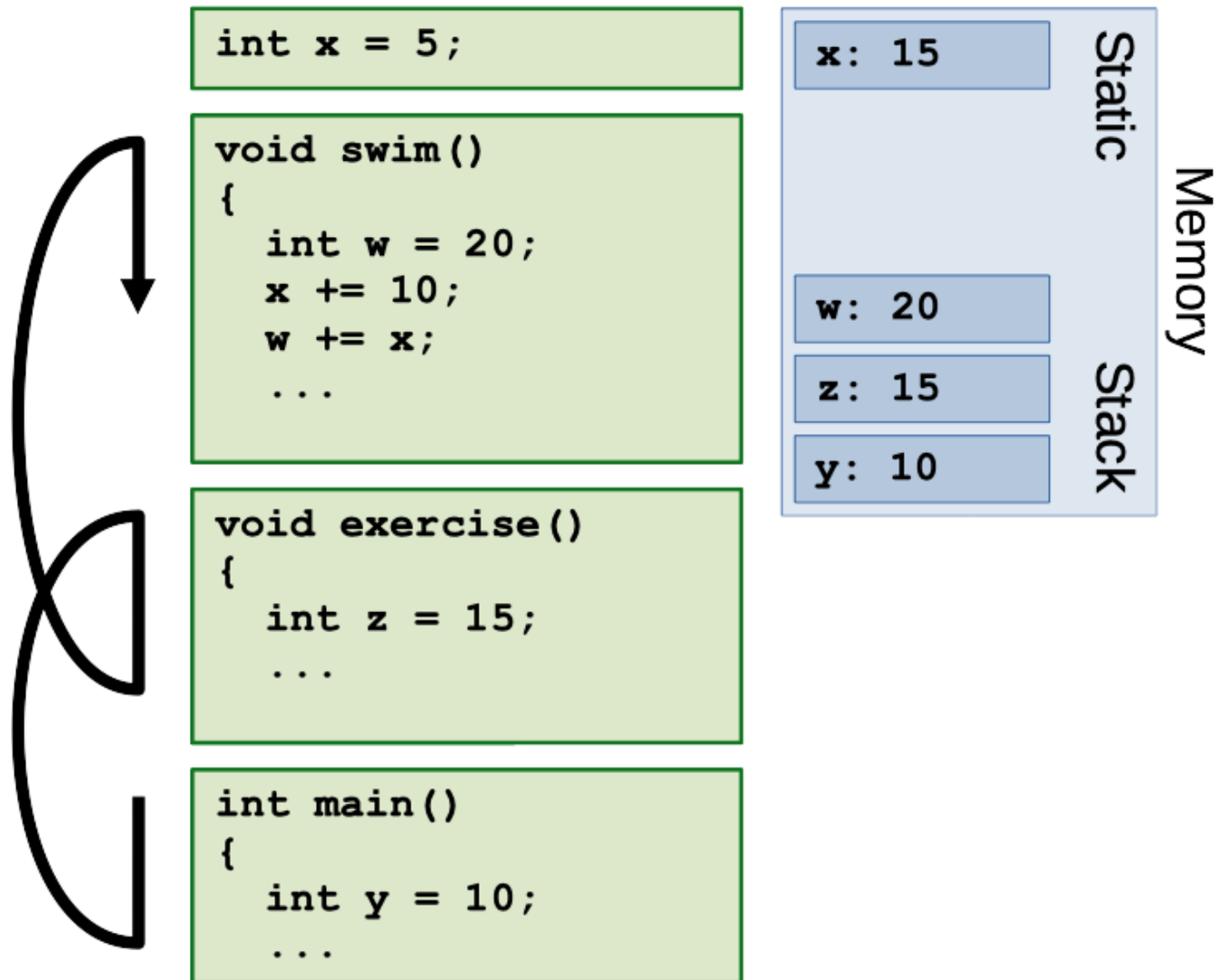
Variable Allocation/Initialization



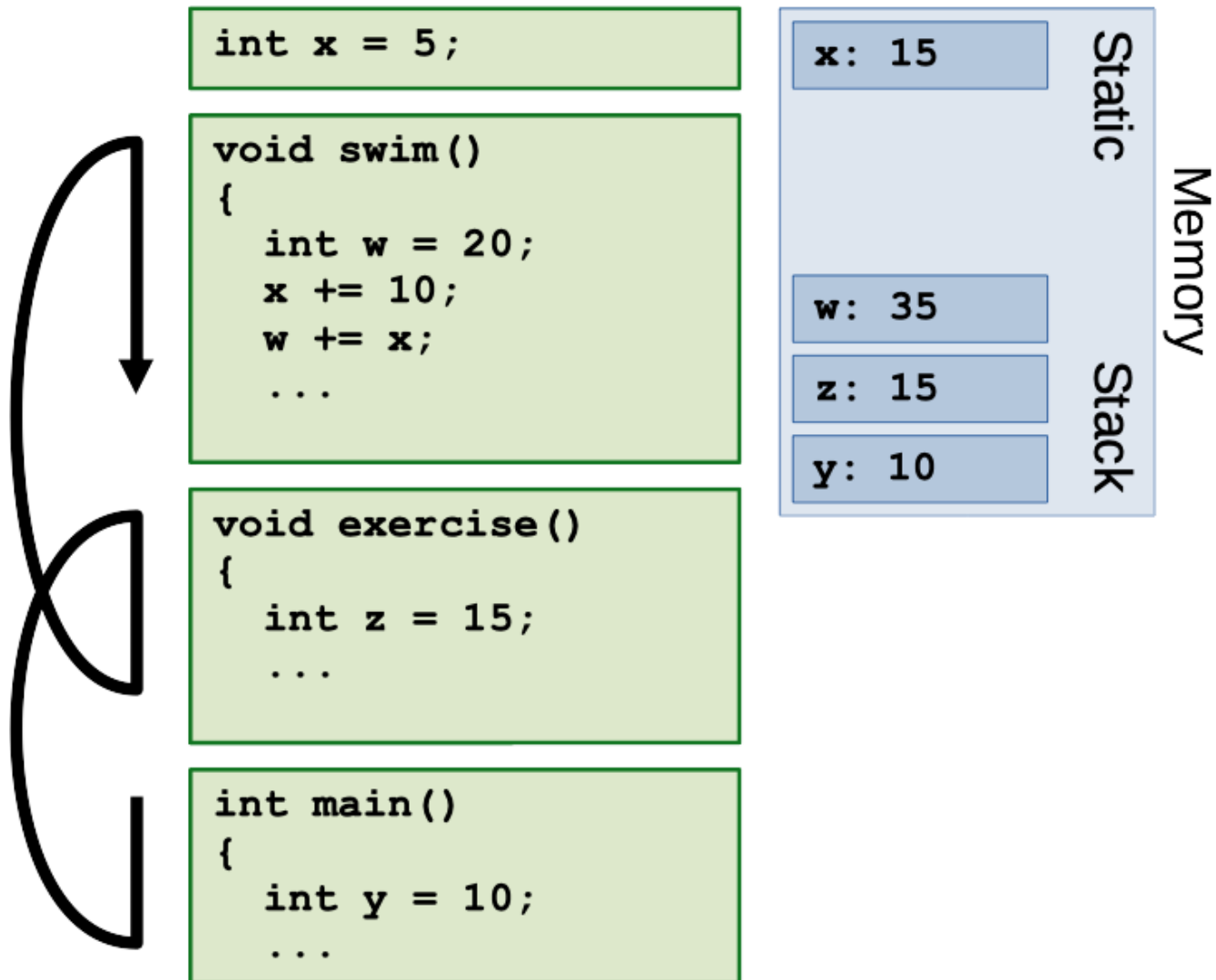
Variable Allocation/Initialization



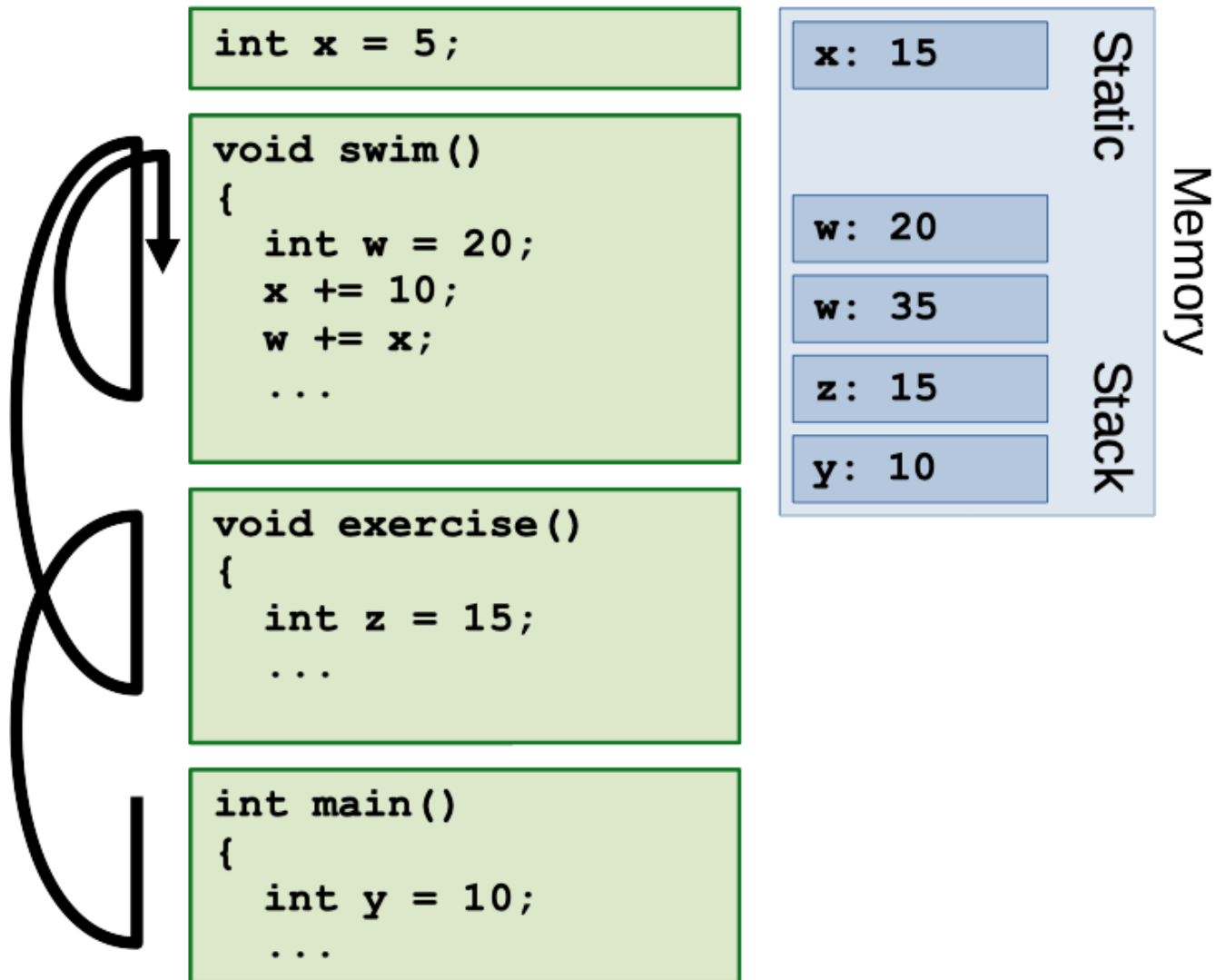
Variable Allocation/Initialization



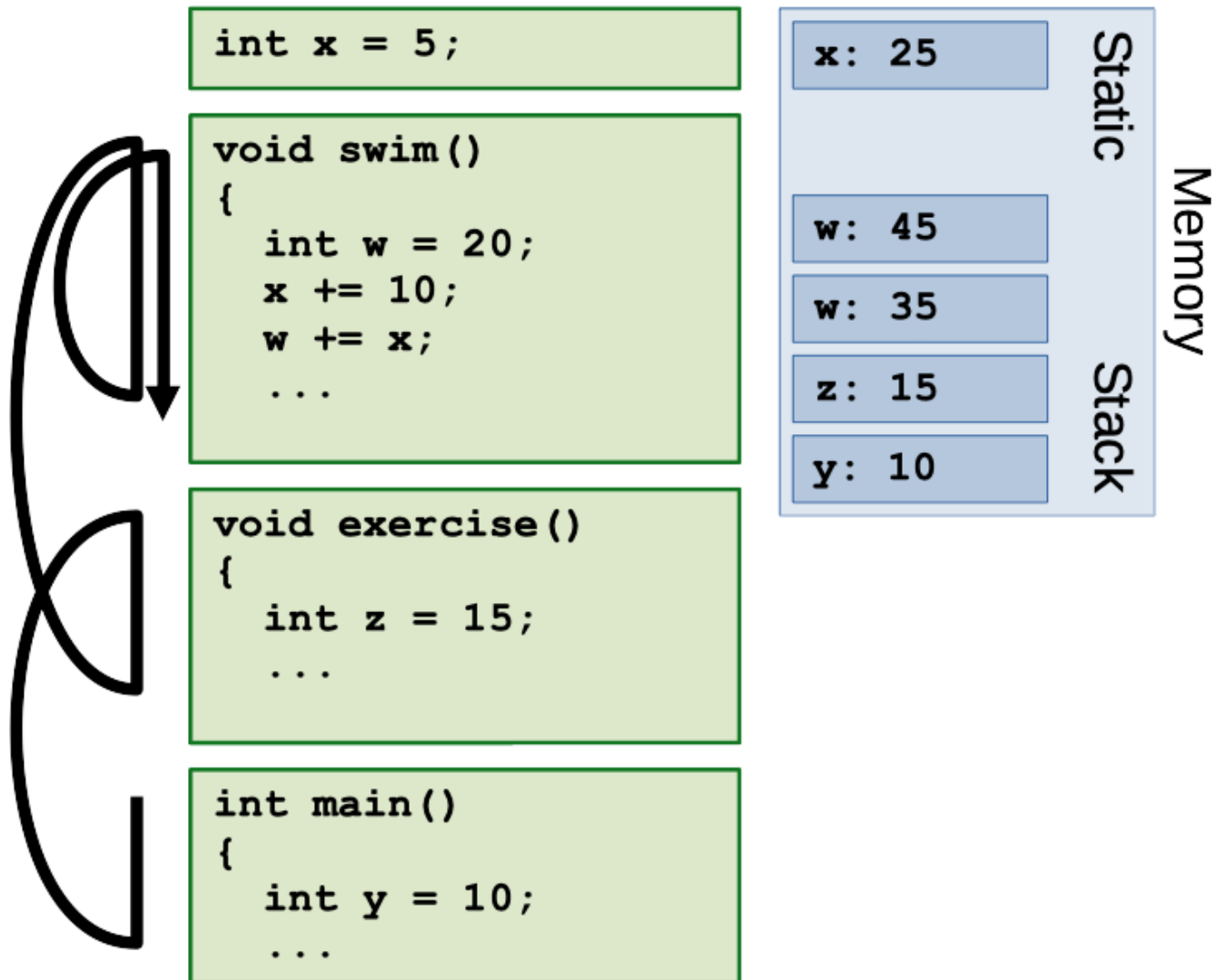
Variable Allocation/Initialization



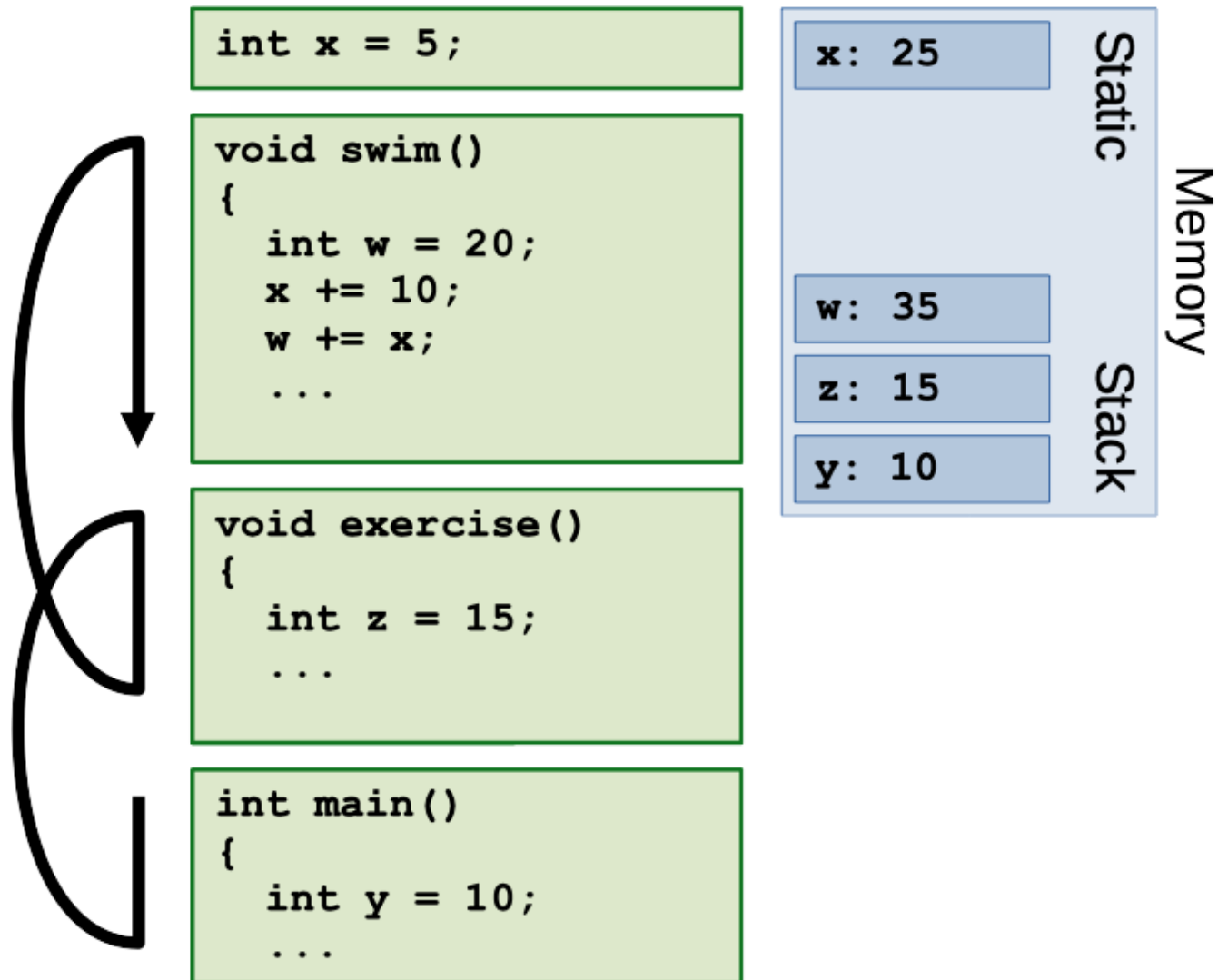
Variable Allocation/Initialization



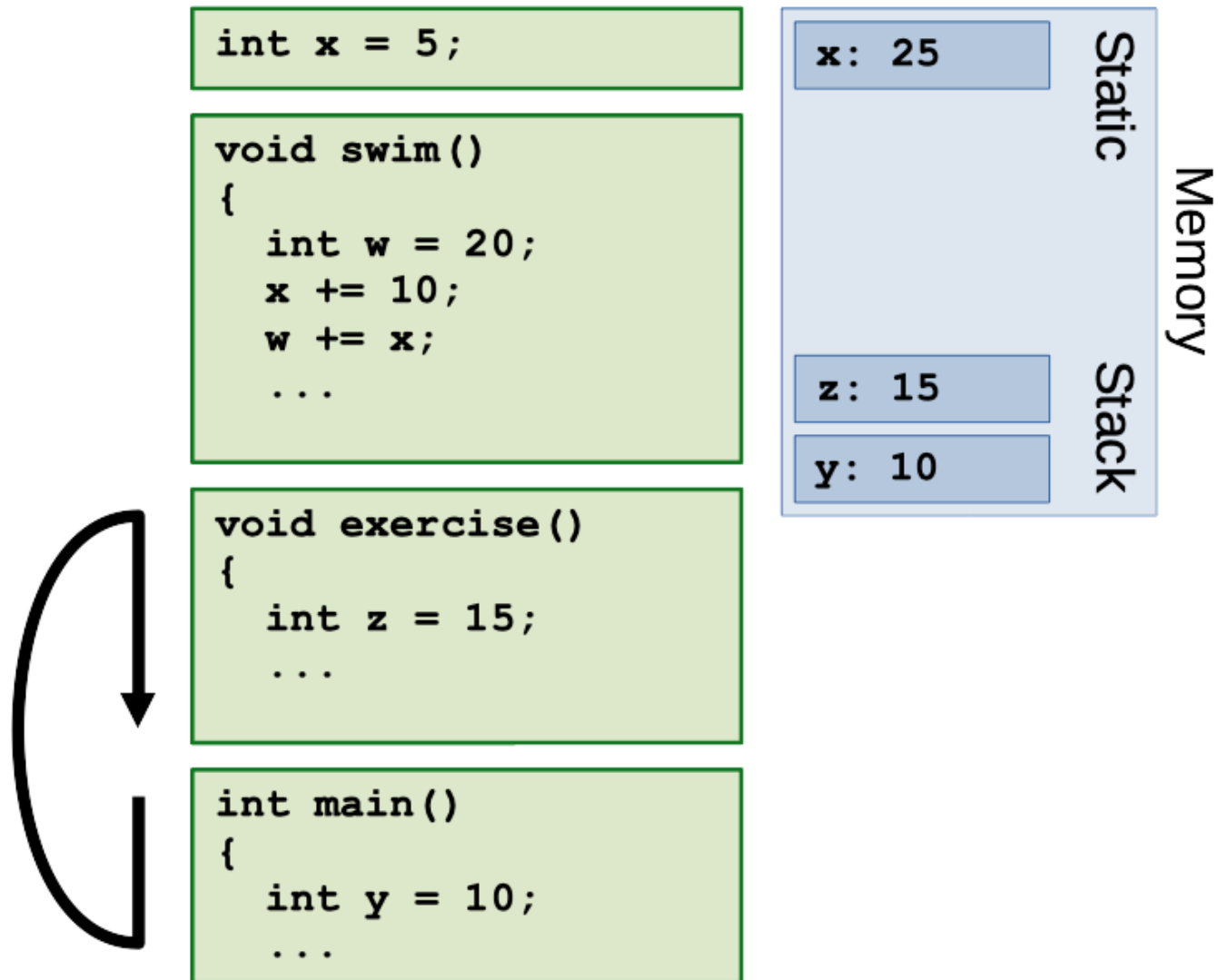
Variable Allocation/Initialization



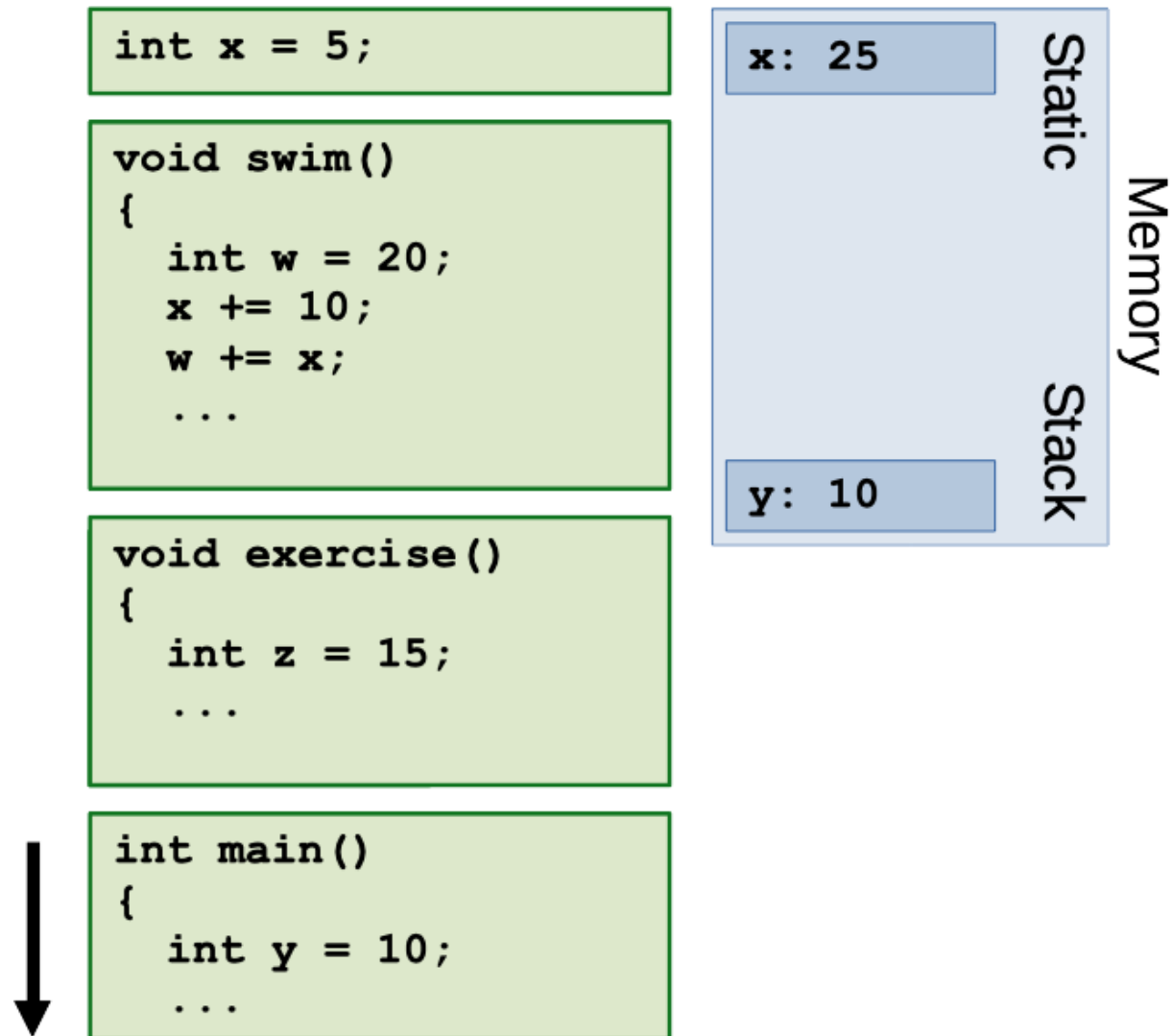
Variable Allocation/Initialization



Variable Allocation/Initialization



Variable Allocation/Initialization



Static Local Variables

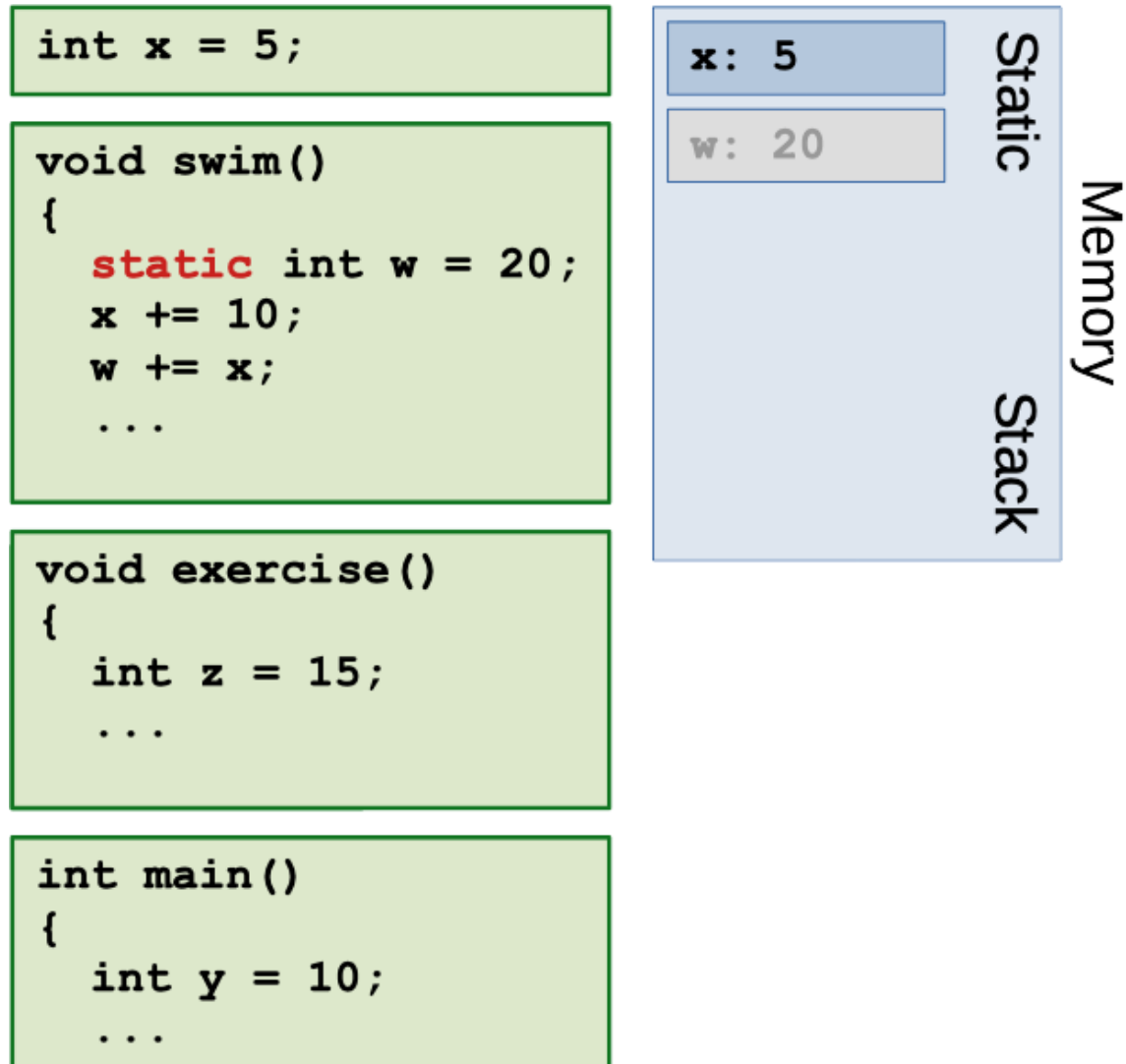
- Scope isn't the same as storage class.
 - You can have local variables that are statically allocated
 - How? Using the keyword `static`.

```
for ( int i = 0; i < 100; i++ ) {  
    static int a = 10 + EXTRA;  
    ...;  
}
```

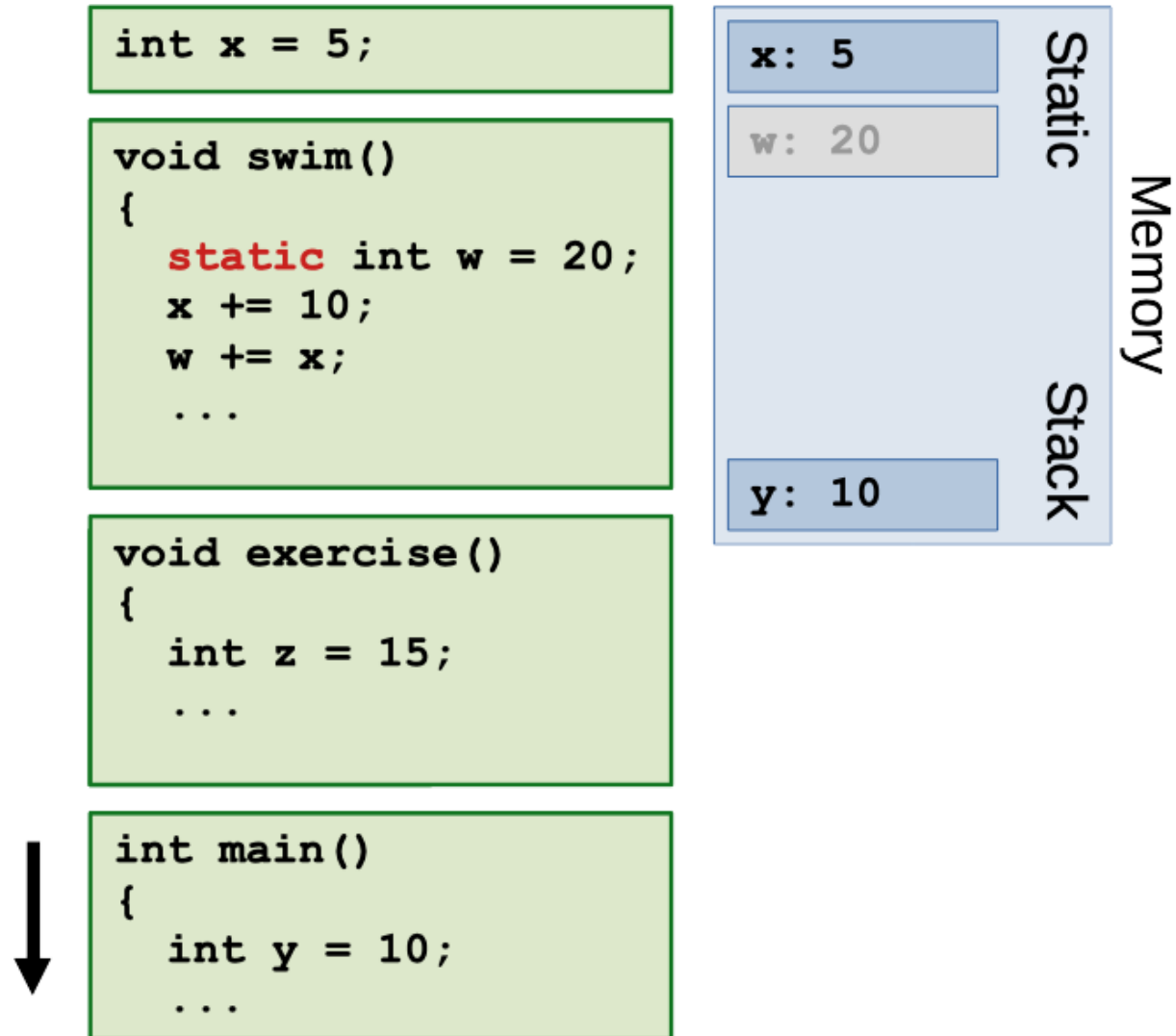
I just get created and initialized once.

- It's like a variable that has the lifetime of a global, but the scope of a stack variable.
- Like a global, requires a **constant initialization expression**.

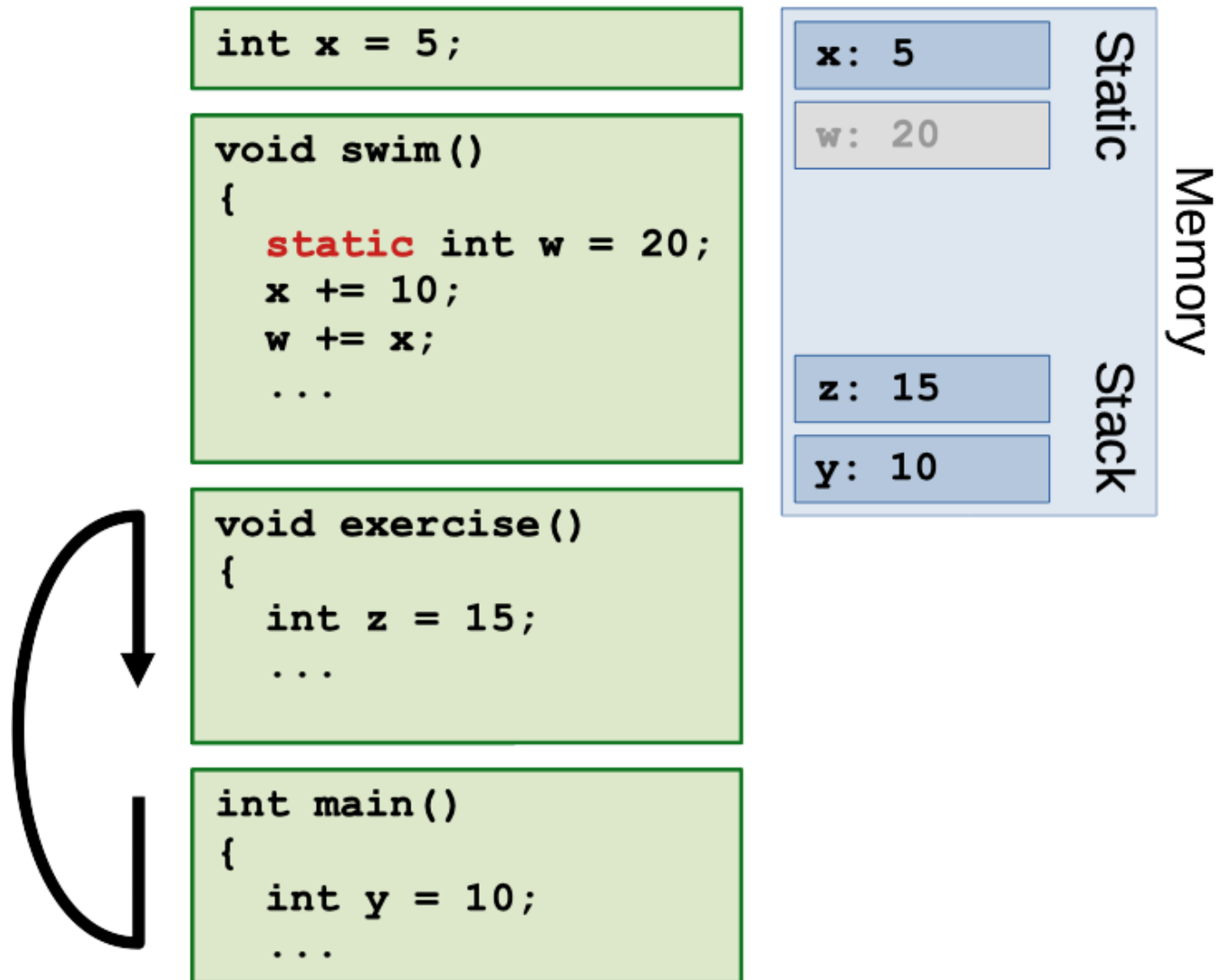
Variable Allocation/Initialization



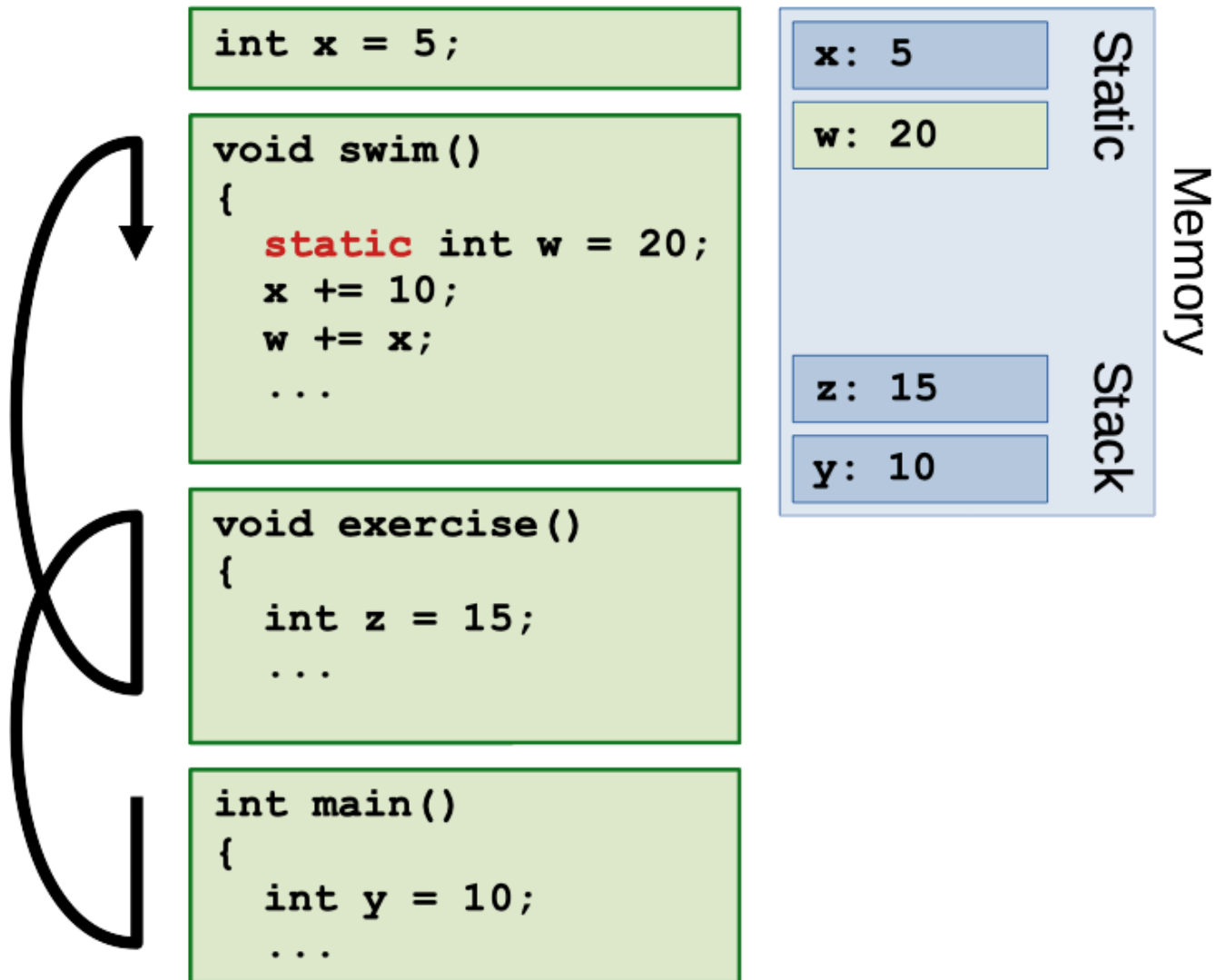
Variable Allocation/Initialization



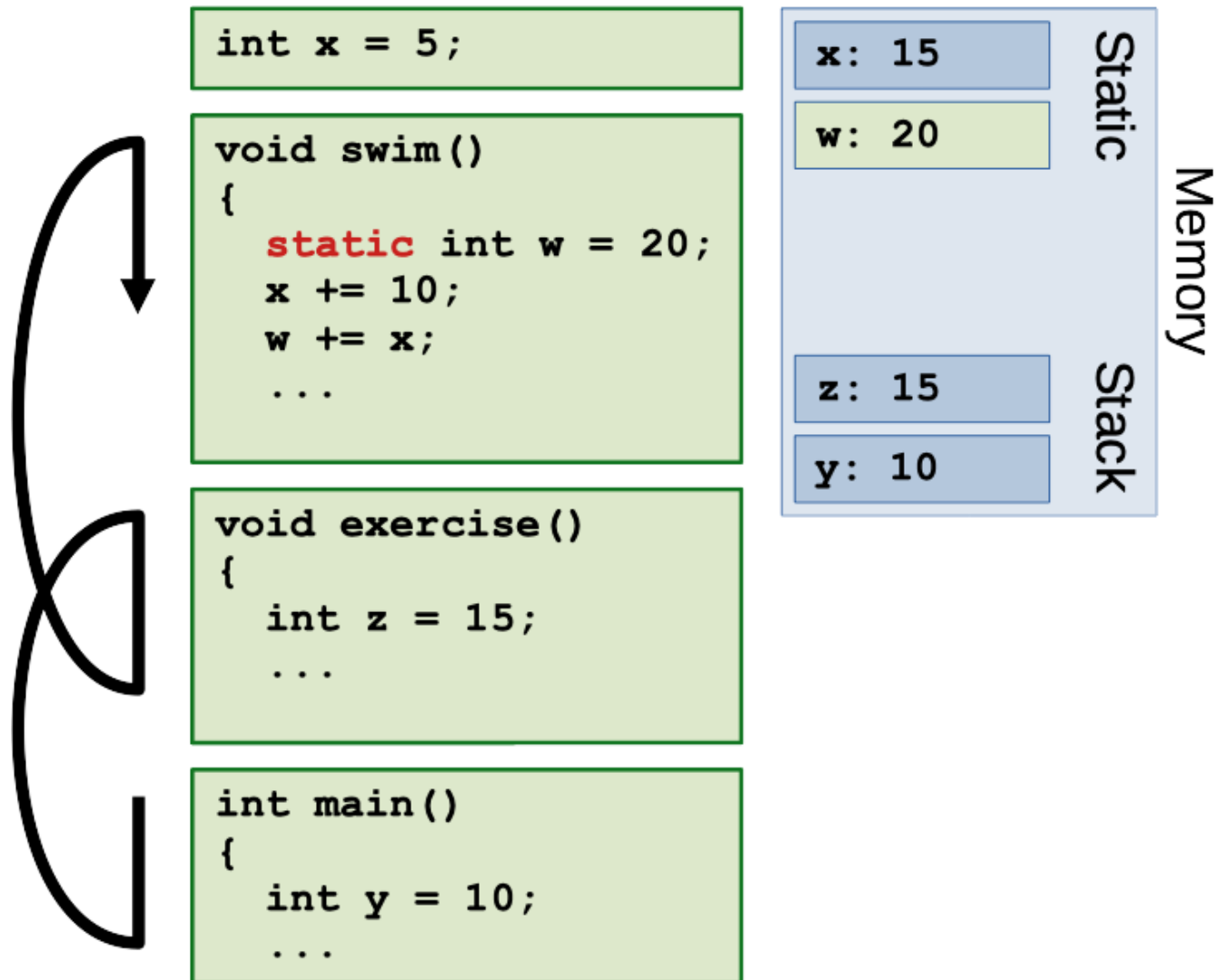
Variable Allocation/Initialization



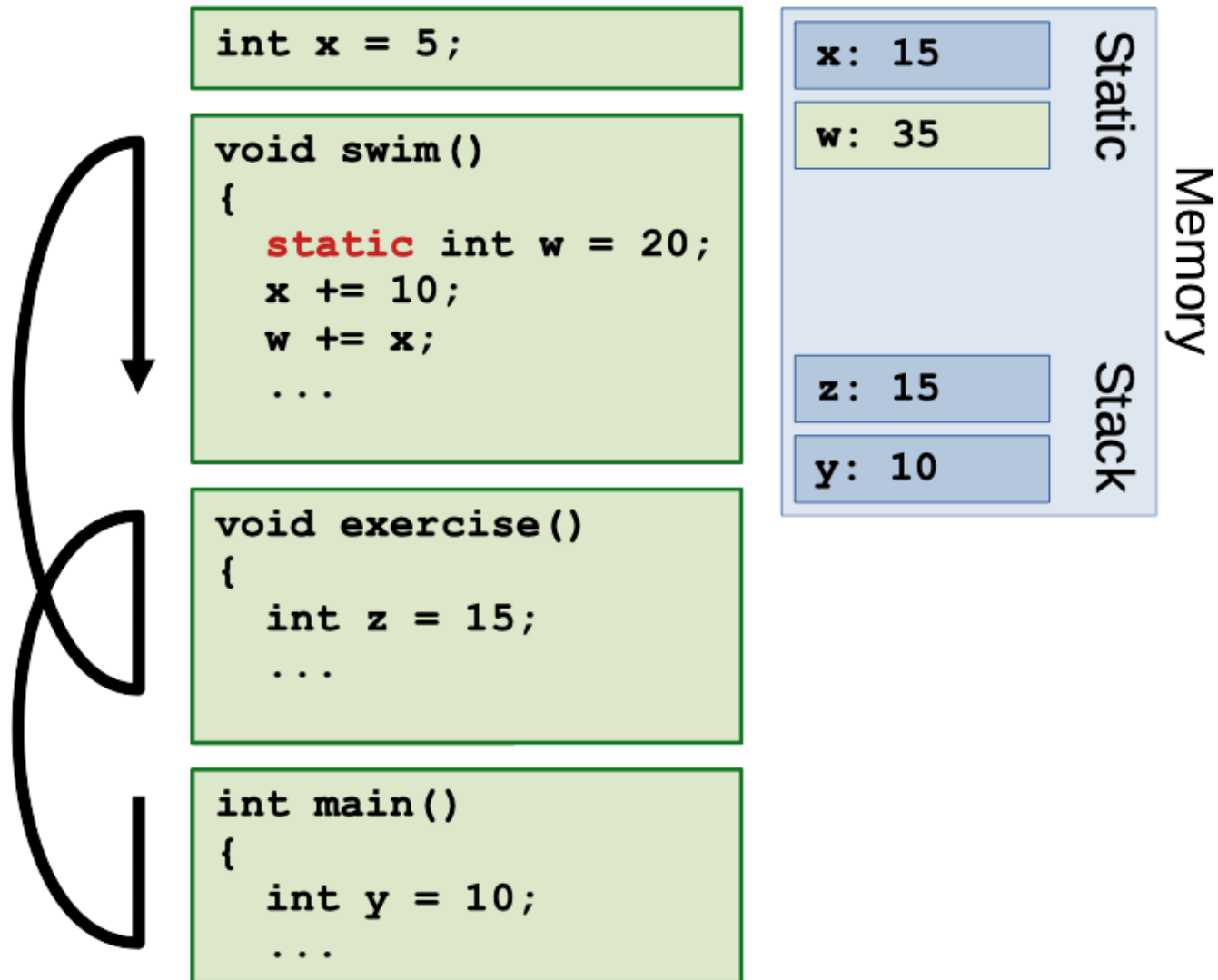
Variable Allocation/Initialization



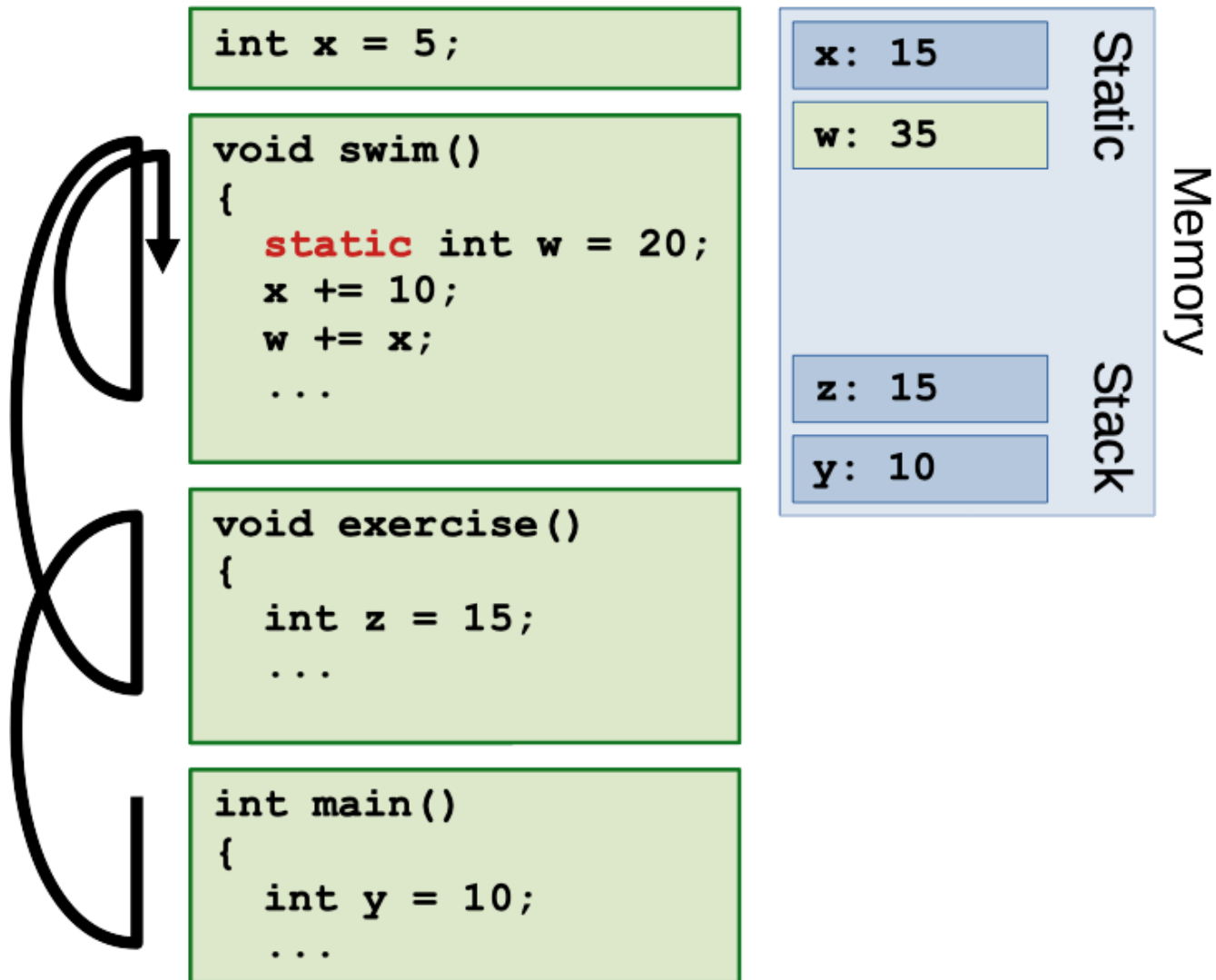
Variable Allocation/Initialization



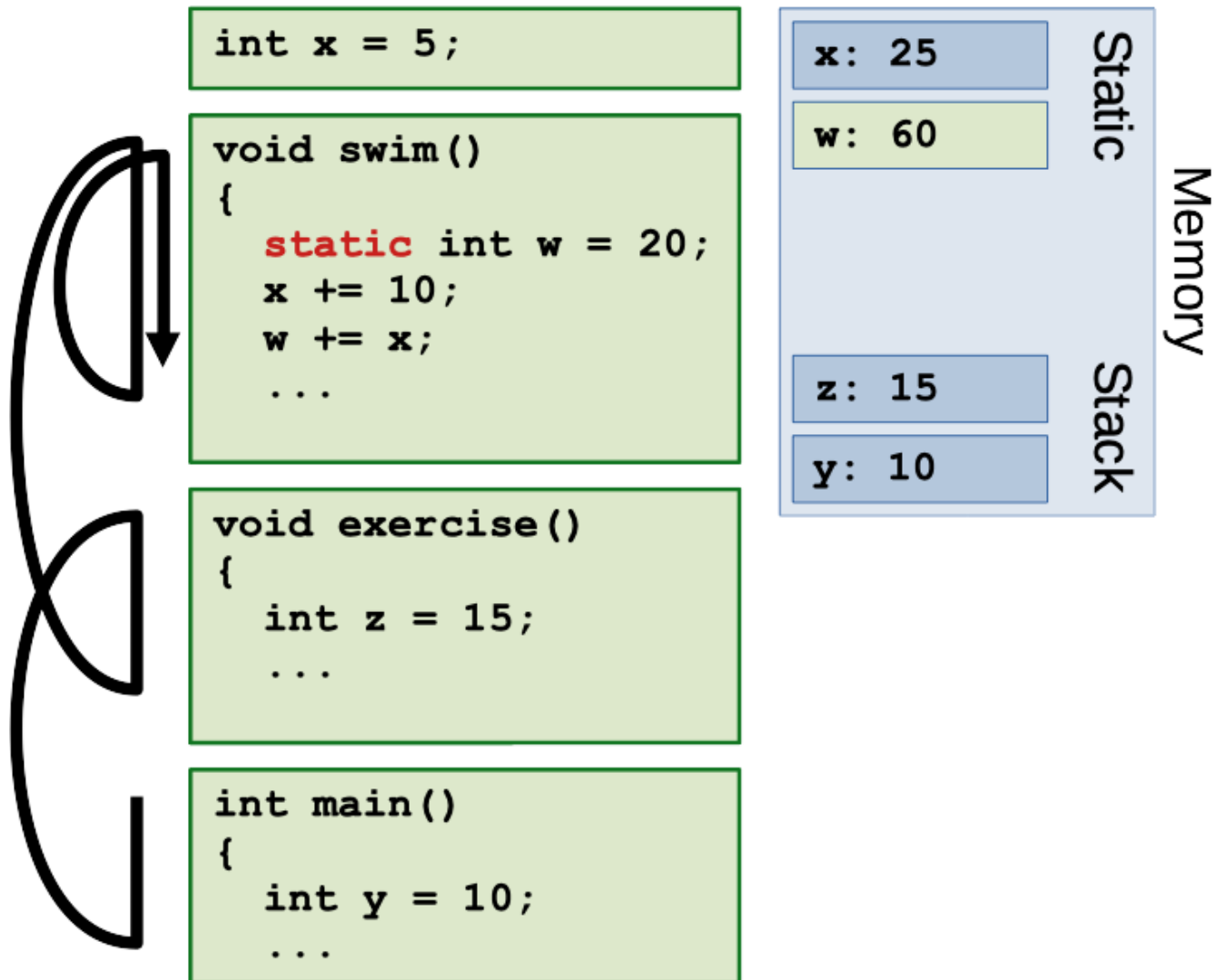
Variable Allocation/Initialization



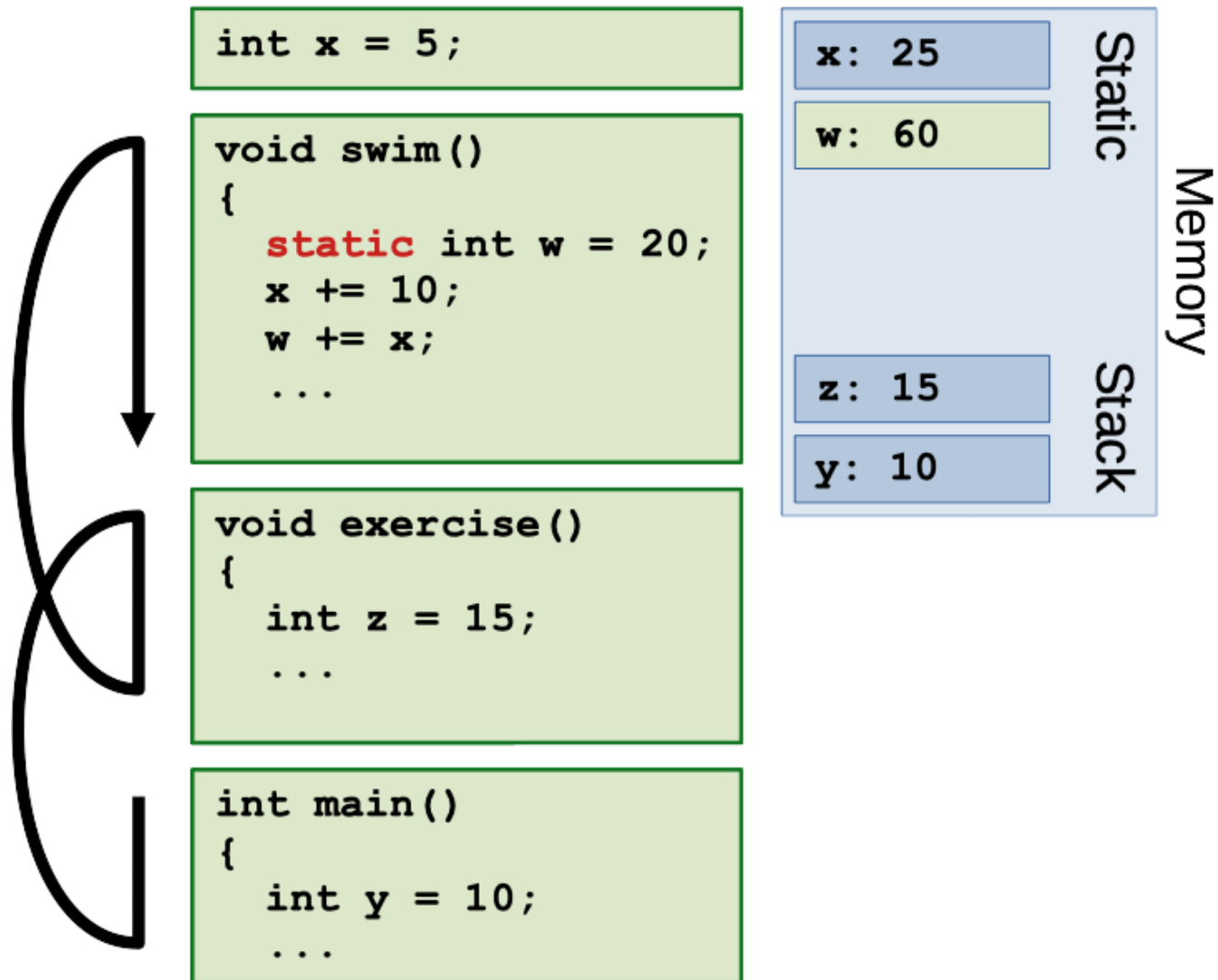
Variable Allocation/Initialization



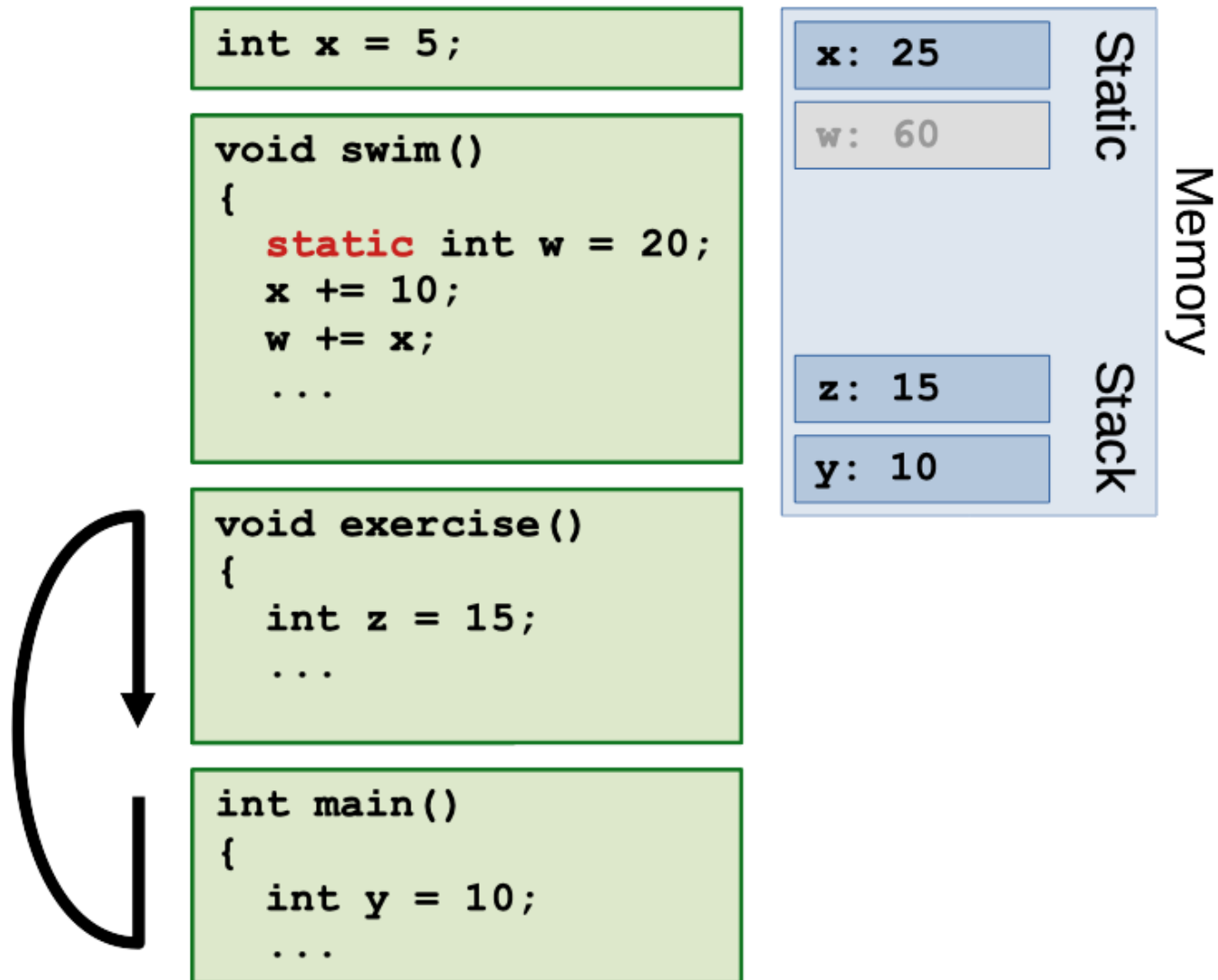
Variable Allocation/Initialization



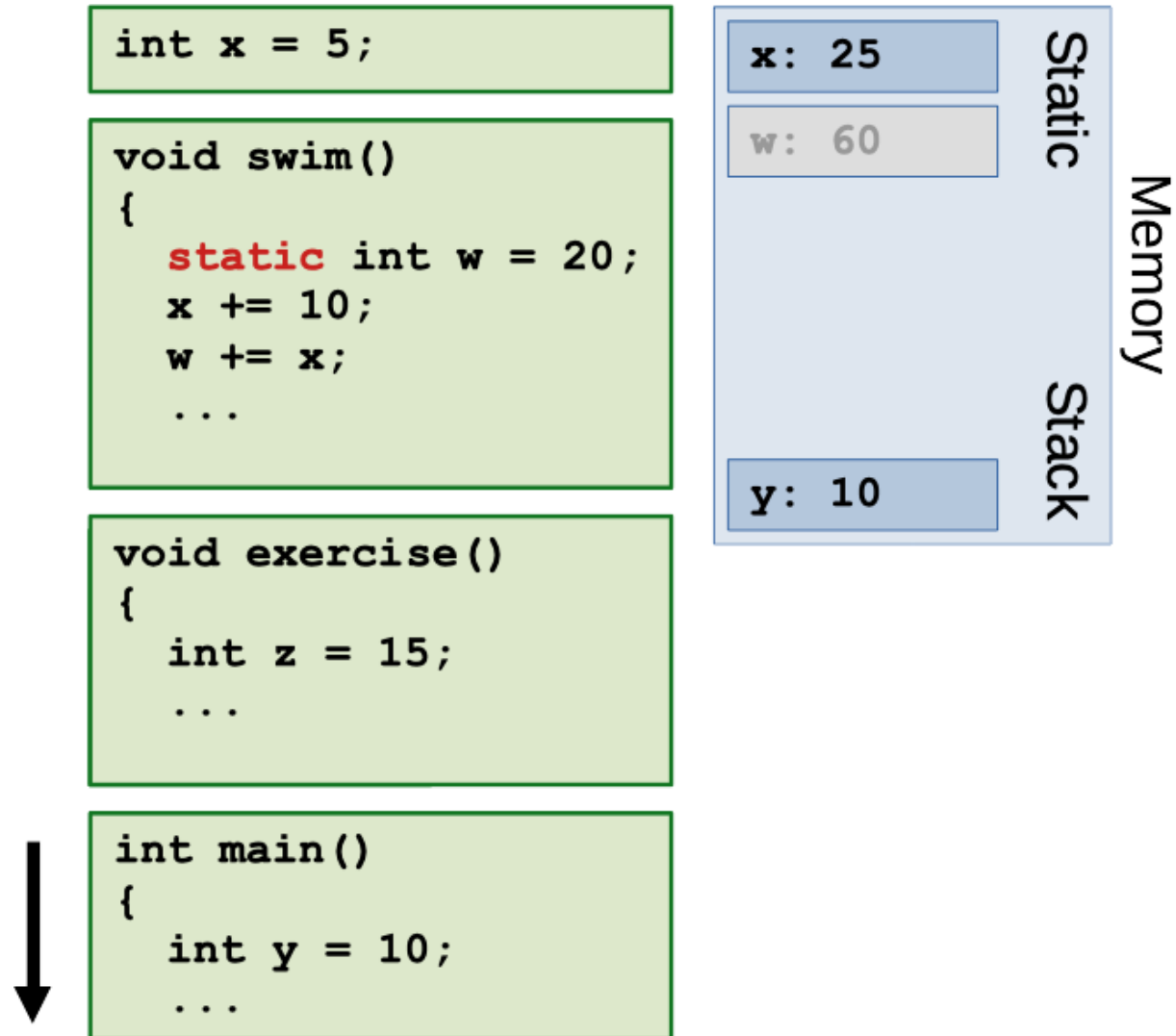
Variable Allocation/Initialization



Variable Allocation/Initialization



Variable Allocation/Initialization



Variable Initialization Summary

- Local variables
 - Initialized by re-evaluating their initialization expression each time they are created.
 - Or, not initialized at all if there's no initialization expression.
- Global variables
 - Initialized once at the start of execution via their constant initialization expression
 - Or initialized to zero if you don't supply an expression.
- Static local variables
 - Just like global

Designing with Functions

- While a Java program is a collection of cooperating objects
- ... a C program is a collection of cooperating functions.
- Functions are our best building block for a good design.
 - It's how we can share code across multiple parts of an application.
 - It's how we re-use code across multiple projects.
 - A library is mostly a collection of functions we can call.

Function Parameters

- Functions can take any number of parameters
- Including no parameters

– Like

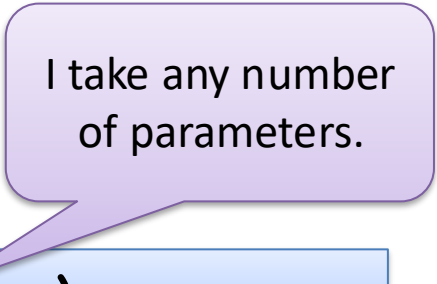
```
void abort( void )
```

This says “I don’t take any parameters”.

A function to immediately exit, unsuccessfully.

Parameter Matching

- To call a function, you need to provide
 - The right number of actual parameters
 - ... and they have to be convert-able to the formal-parameter types
- You can define functions that take any number of parameters of various types.
 - We have a ... notation for this.
 - ... more on this later.



I take any number of parameters.

```
void printFloats( int n, ... );
```

Function Return Values

- A void return type says the function doesn't return anything.

```
void printFloat( float x );
```

- Or, a function can return a value of (just about) any type.
- It sure would be nice to be able to return more than one value.
 - But, as in Java, each function has one precious return slot.
 - Soon, we'll learn a few tricks to get around that.

Parameter Conversion

- C will implicitly convert *actual parameters*
- ... to the *formal parameter* types

```
int expunge( float x, long y )  
{  
    ...  
}
```

I'm a formal parameter, x.

These will convert to float and long to make the call.

```
int a = 7;  
char b = 'm';  
expunge( a, b );
```

I'm an actual parameter, 7.

Parameter Conversion

- This is why the compiler wants to know about a function before it gets called.
 - It needs to know what to convert the parameters to.

```
int a = 7;  
char b = 'm';  
expunge( a, b );
```

I guess maybe this function takes two ints?

Oops. I was wrong. Too late to fix it now.

```
int expunge( float x, long y )  
{  
    ...  
}
```

Knowing a Function's Type

- The compiler can learn about a function's type (return and parameter types) without seeing the whole function definition.

```
int expunge( float x, long y )  
{  
    ...  
}
```

I'm a
definition.

- This is done with a *function prototype*

```
int expunge( float x, long y );
```

I'm just a
prototype.

Types in Prototypes

- A prototype just needs the typing information.

```
int expunge( float, long );
```

- But, you can include parameter names if you want.

```
int expunge( float x, long y );
```

- Having the names can help with documentation.

Definition vs Declaration

- This demonstrates the difference between a definition and a declaration.
- *Definition* : here's some thing, allocate space or write code for it Mr. Compiler
 - Like a function definition.
- *Declaration* : there's something like this out there, but it's defined elsewhere.
 - You may see me trying to use it, so don't act surprised Mr. Compiler.
 - Like a function prototype.

Code Organization

- With prototypes, we can define our functions in any order.
- As long as we prototype them before they're used.

There's a function like this.

```
int expunge( float x, long y );
```

OK. I've heard of this function before.

```
int a = 7;  
char b = 'm';  
expunge( a, b );
```

OK. Just like I expected.

```
int expunge( float x, long y )  
{  
    ...  
}
```


Extern Variables

- That's what the `extern` keyword is for
 - It tells the compiler about a something that's defined elsewhere.
 - It lets us describe a global variable without allocating space for it.
 - So, it lets us declare a variable without defining it.

```
int x;
```

I'm a declaration **and** a definition.

```
extern int x;
```

I'm just a declaration.

Extern and Prototypes

- This is the same kind of thing a prototype does.
 - It tells the compiler about something defined elsewhere (or later).
- In fact, you can use the keyword `extern` with a prototype

I'm a declaration **and** a definition.

```
void f( int a ) {  
    ...  
}
```

I'm just a declaration.

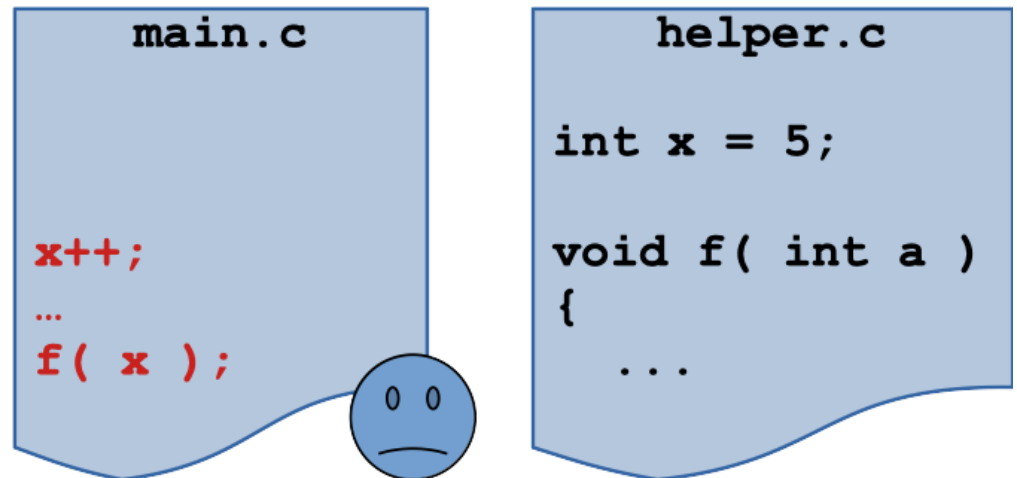
```
extern void f( int a );
```

I'm optional.

- But, `extern` doesn't change anything on a prototype, so most people don't use it here.

Multi-File Programs

- We can make **functions** and **global variables** that are **visible to multiple source files**.
- But, the compiler looks at each source file as an individual ***compilation unit***.
 - it won't let us use symbols it has never heard of.



Using Extern Declarations

- We can use extern variable declarations and prototypes to access symbols (variables and functions) defined elsewhere.
 - This makes the compiler happy when it sees us use them.
 - Then, the linker connects each use with its memory location.

main.c

```
extern int x;  
void f( int );  
  
x++;  
f( x );
```



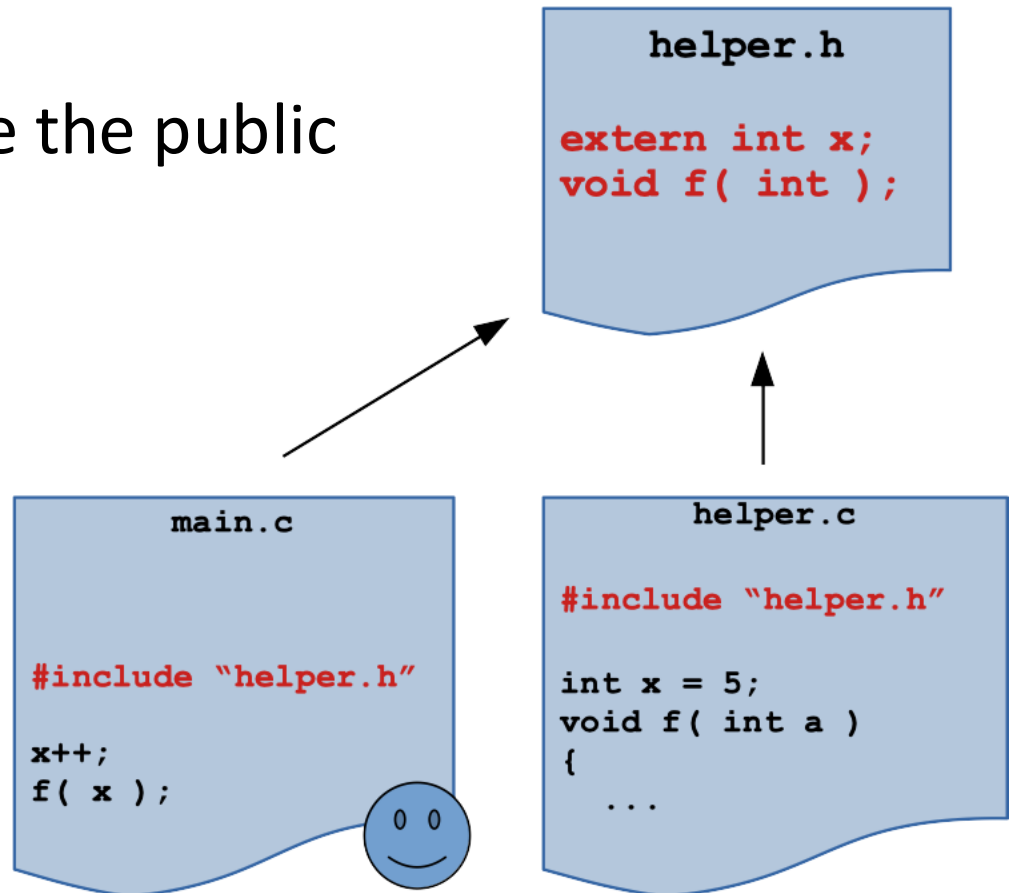
helper.c

```
int x = 5;  
  
void f( int a )  
{  
    ...  
}
```

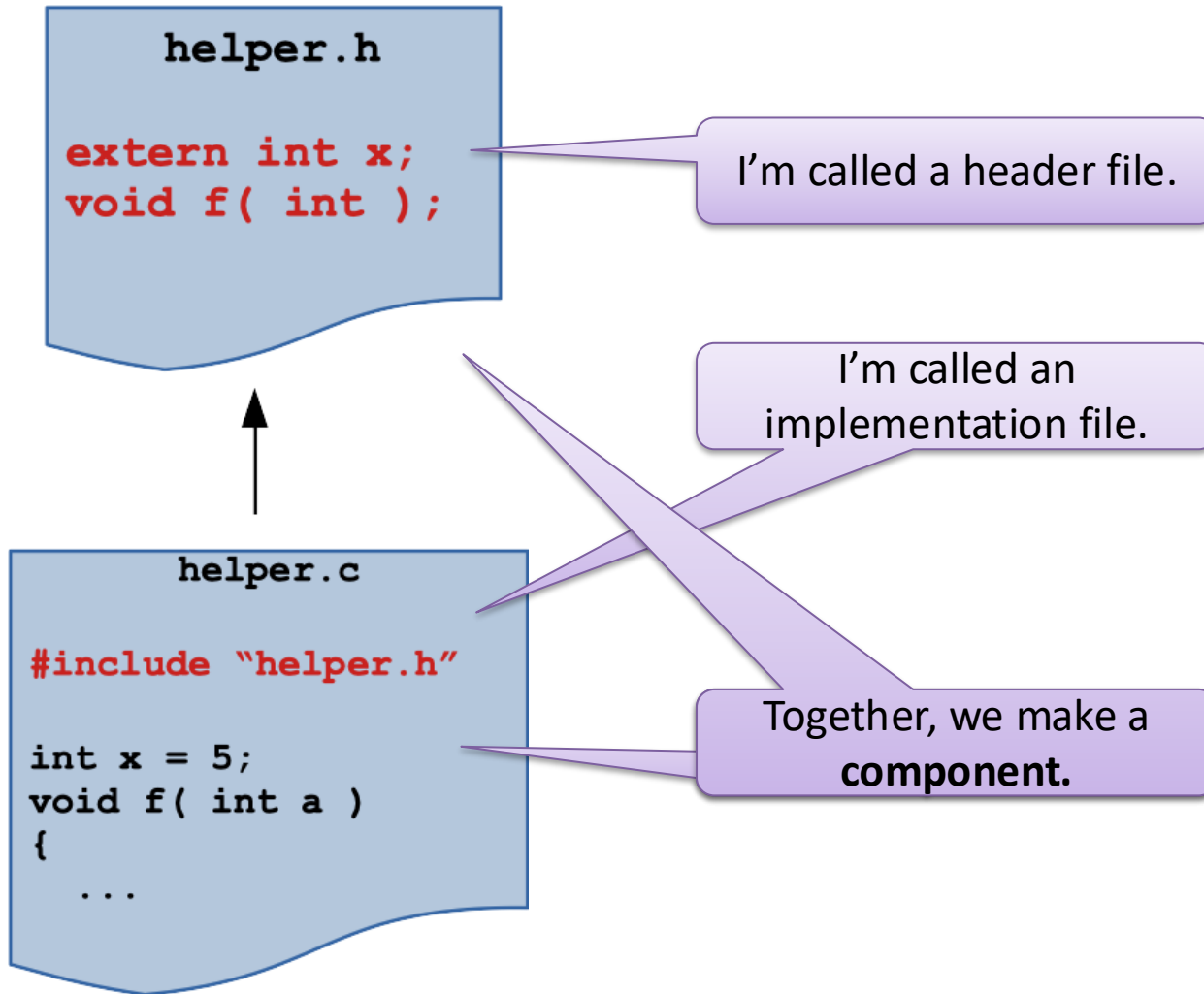
I'm still not
completely happy.

Using Headers

- Really, this the right job for a **header file**.
 - It should advertise the public interface for an **implementation file**.

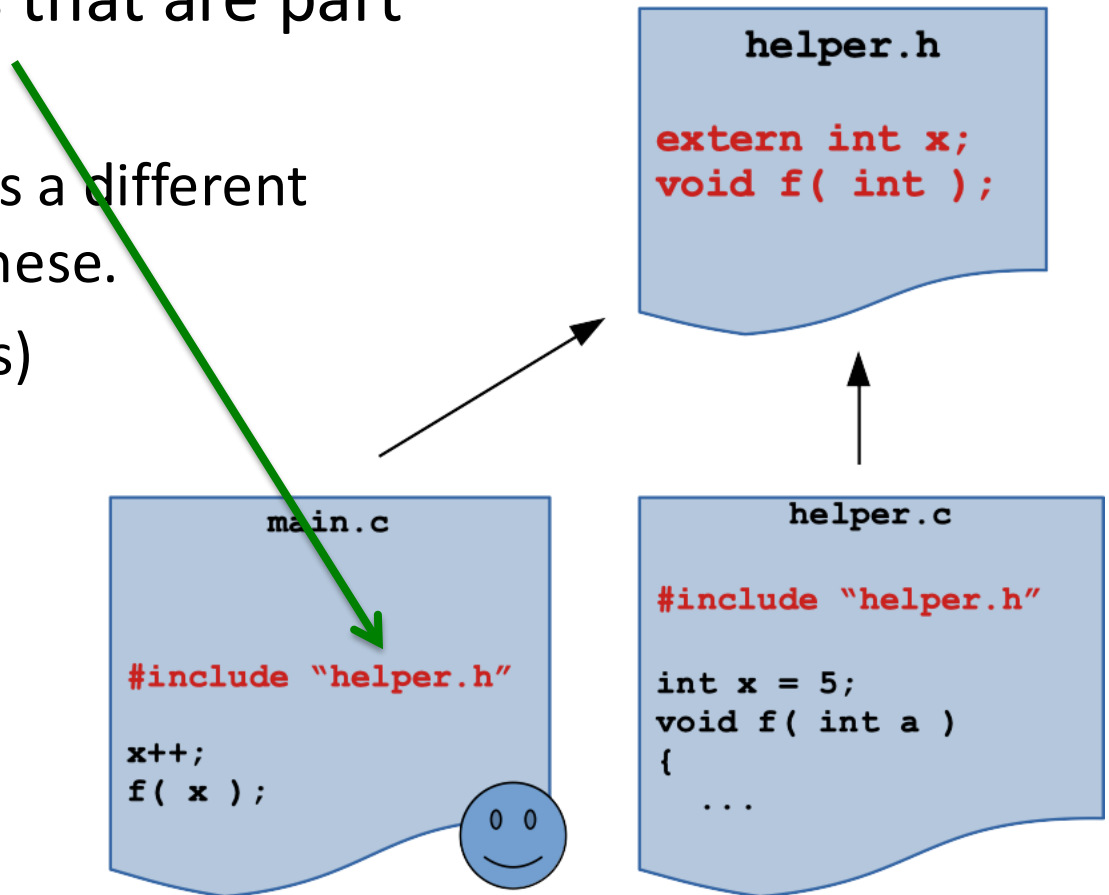


Components



Things to Notice

- We use double quotes when including headers that are part of our project.
 - The compiler uses a different search path for these.
 - (vs angle brackets)



Things to Notice

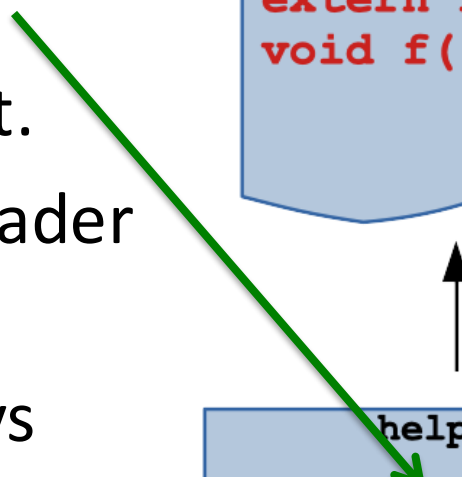
- It's a good practice to include your own header file first.
 - You may not actually need it.
 - ... but it makes sure your header compiles cleanly.
 - ... and its declarations always match what you're defining.

helper.h

```
extern int x;  
void f( int );
```

helper.c

```
#include "helper.h"  
  
int x = 5;  
void f( int a )  
{  
    ...  
}
```



Keeping Things Private

- There's another use of the `static` keyword.
 - For global variables, it marks them as invisible to the linker.
 - This works for functions also.
- We say it gives these symbols *internal linkage*
 - Good for symbols private to one component.
 - Like a private static field in Java.

main.c

```
extern int x;  
x++;  
  
void f( int );  
f( );
```



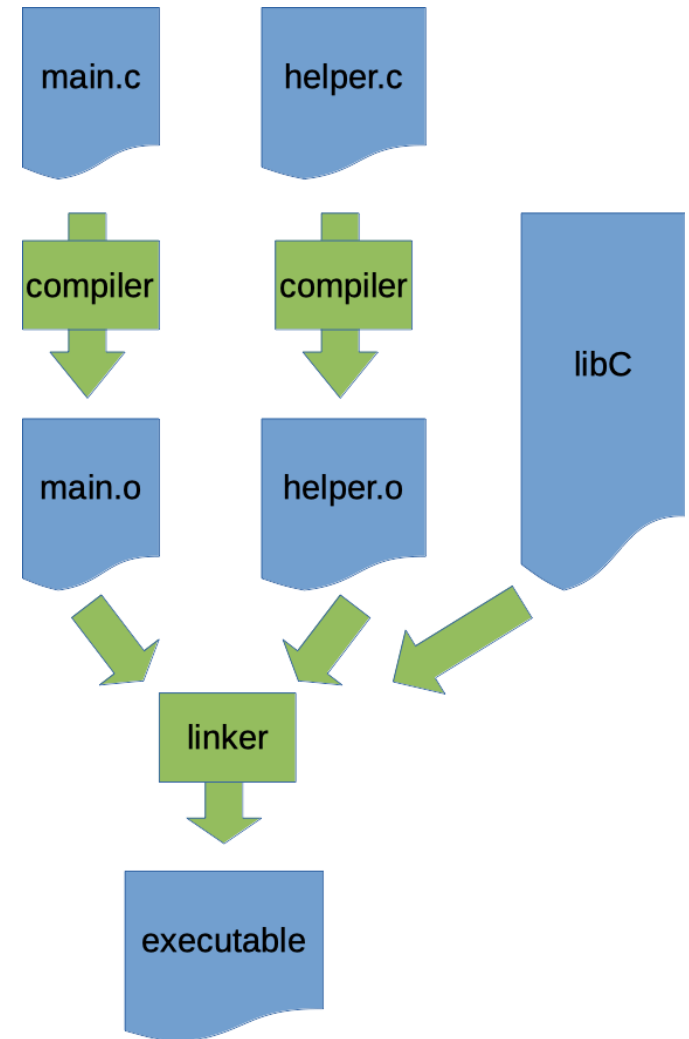
helper.c

```
static int x = 5;  
  
static void f( int a ) {  
    ...;  
}
```



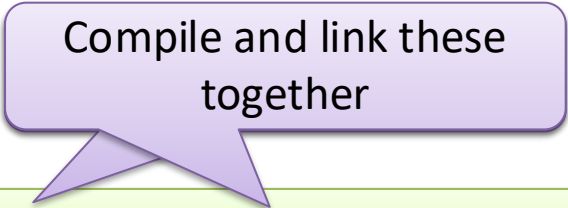
Building a Program

- Building a program requires two steps.
 - **Compiling** each implementation file to an **object file**.
 - **Linking** all the objects together into a single **address space**.
- The linker hooks up uses of symbols with their definitions in other components.
- Even if we only have one component, we still need to link with any libraries we use.



Compiling and Linking

- gcc can compile and link everything with one command.



Compile and link these
together

```
$ gcc -Wall -std=c99 main.c helper.c
```

- This is an easy way to build a multi-file program ... but it's not very efficient.
- Mistakes in a header file or the gcc command can produce new types of errors ...

Compile-Time Errors

- The compiler will complain if it sees you using a symbol it's never heard of.

```
...  
f( x );  
...
```

This is a complaint
from the compiler.

```
eos$ gcc -Wall -std=c99 program.c  
program.c: ... warning: implicit declaration of function 'f' ...
```

- Maybe you forgot to include a header you need.
- Or, you forgot a prototype or extern variable declaration in a header.

Link-Time Errors

- With a prototype or an extern variable declaration, the compiler won't complain.
- But, the linker will if it can't find a definition for the function.

```
...  
f( x );  
...
```

This is a complaint
from the linker.

```
eos$ gcc -Wall -std=c99 program.c  
test.c:...: undefined reference to `f'
```

- Maybe you forgot to compile/link with one of the implementation files.